Training a biomechanical model of a human to walk using a genetic algorithm

Daniil Bastrich

Sep 2024

Table of contents

1. Introduction

The source code is available at https://github.com/bastrich/ga-walking.

The idea is to train a biomechanical model of a human body to walk using a genetic algorithm. I will use an OpenSim environment [5] with a model consisting of human bones connected with muscles. My evolutionary computing algorithm will explore the space of combinations of muscle activations to find an optimal walking strategy in several different scenarios.

1.1. Project objectives

1. **Teach a model to walk in 2D space.**
   This is the starting objective, which serves at least two purposes:
   a. Increase the level of confidence in the feasibility of the project.
   b. Prepare the initial state for training a model to walk in 3D space. Hypothetically, teaching a model to walk in 3D space is expected to be easier when I have the model already walking in 2D because I can reuse it.

   The definition of done for this objective is a model walking in 2D space for the whole provided simulation time and having visual subjective similarity to natural human walking. Defining some strict formal criteria here is difficult, but I will try to provide them based on a fitness function in 3. Design.
2. **Experiment with teaching a model to walk in 3D space.**
   In this objective I will try to train a model to walk in 3D space, experimenting with different approaches, including starting from scratch or reusing the results from the 1st objective, higher resolution of the action space, and others.

1.2. Motivation

1. Discovering new options to solve AI problems without deep learning using evolutionary algorithms. In cases when there is no learning data, or it is difficult to define them, when there is no strict definition of correct/incorrect solutions, but there is a correct direction and some formal definition for that direction, evolutionary computing may be more suitable than traditional statistical learning algorithms. In some cases, it also requires less memory and computational resources. According to [10], Evolution Strategies are often underestimated, and this project aims to show how genetic algorithms can be used for such kinds of tasks.
2. Better understanding of how the human body works when walking. As a result of the project evaluation, it will be possible to visualize the work of muscles while walking in real time and use that knowledge in biological and anthropological research. The resulting predictive simulation may also be used to speed up medical research, such as assistive devices or rehabilitation treatments.
3. Better understanding of human development stages which lead to knowing how to walk. We all learn how to walk, whether in childhood or after serious injuries. This project may show how that skill is acquired step by step and can be used in pediatric medicine or rehabilitation.

2. Literature review

2.1. Genetic Algorithms

[1] is a classic work about Genetic Algorithms that describes the creation of virtual 3D creatures of different forms and training them in specific skills like walking, swimming, and so on. The author uses an approach inspired by biological evolution to develop such creatures. The article presents a formal genetic language for defining a genotype, rules for phenotype representation of a specific genotype, simulation environment and corresponding fitness function, and mutability rules. Using those building blocks, the evolutionary genetic algorithm producing remarkable working creatures is implemented. See **Figure 1** for example.

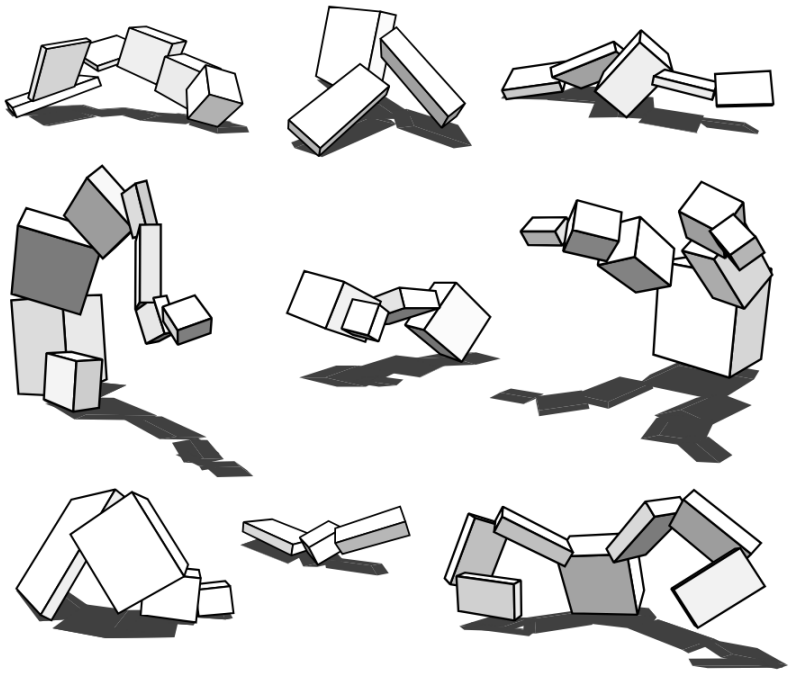

Figure 1. Examples of creatures evolved for walking in [1].

Though it can not be considered a “state-of-the-art” approach in modern times, as more advanced algorithms and simulation environments for specific tasks evolve, it laid a base for many future studies in the field of genetic and evolutionary algorithms and can still be used as a starting point for new research.

My project can be conceptually classified as an automatic design of moving structures with desired features. The article shows an original and effective approach to such tasks.

The paper is a theoretical base for my project as it contains a detailed description of the solution similar to that to be used, including the algorithm and its basic blocks. It shows the feasibility of the current project on the conceptual level. My project already has a predefined creature, but how it moves will be defined by implementing a similar Genetic Algorithm.

Some parts to reuse include

1. The genetic algorithm on a high level.
2. Mutability options: Gaussian-like distribution for alteration of genes, crossover.

Main parts to modify and adjust:

1. The research has a mostly static mutation rate depending only on genotype size. My project will adjust the mutation rate dynamically during evolution in case of a long absence of progress.
2. The research referenced describes simple fitness functions based on distance and/or velocity, but I will develop a more sophisticated one, at least considering a model's foot position and torso angle.
3. The genotype structure in the referenced research is too complicated for my case. So, I will simplify it, leaving only the required parts.

Another useful statement from the article is that it is very important not to have bugs or glitches in a simulation environment and fitness function. Otherwise, the genetic algorithm can exploit them, and the result may be far from expected. At the same time, such a phenomenon can also be used for testing purposes, but not too efficiently. To address such issues, I will cover my code with unit tests.

2.2. Learning To Walk

[2] discusses and analyzes the problem of human locomotion control in neuromechanical simulation. The authors provide a brief overview of human walking, including its neurological and biological aspects. They analyze which AI techniques can be used to build optimal strategies for different kinds of human movement, focusing mostly on Reinforcement Learning, though many aspects still may be useful for Genetic Algorithms as well. The article also describes and summarizes the results of the AI competitions the authors ran.

According to the article, locomotion, and human walking in particular, may be derived based on three hypotheses:

1. "*... locomotion in many animals can be interpreted as a hierarchical structure with two layers, where the lower layer generates basic motor patterns and the higher layer sends commands to the lower layer to modulate the basic patterns*" [2].
2. "*... the lower layer seems to consist of two control mechanisms: reflexes and central pattern generators (CPGs)*" [2], where reflexes provide Feedback Control, i.e. signals based on sensors and CPGs provide Feedforward Control, i.e. direct predefined signals.
3. "*... there is a consensus that humans use minimum effort to conduct well-practiced motor tasks, such as walking*" [2].

See **Figure 2** for a visual structural representation of human walking according to these hypotheses.

Also, the authors ran several competitions dedicated to human walking. The "Learn to Move" competition series was held annually from 2017 to 2019:

1. "NIPS 2017: Learning To Run" involved training a human model to run through obstacles in 2D space (but visually, it was 3D).
2. "NIPS 2018: AI For Prosthetics" involved training a human model to walk in 3D space with a prosthetic leg at a desired speed.
3. "NIPS 2019: Walk Around" involved training a human model to walk in 3D space, following the desired velocity vector, which changed during the simulation.

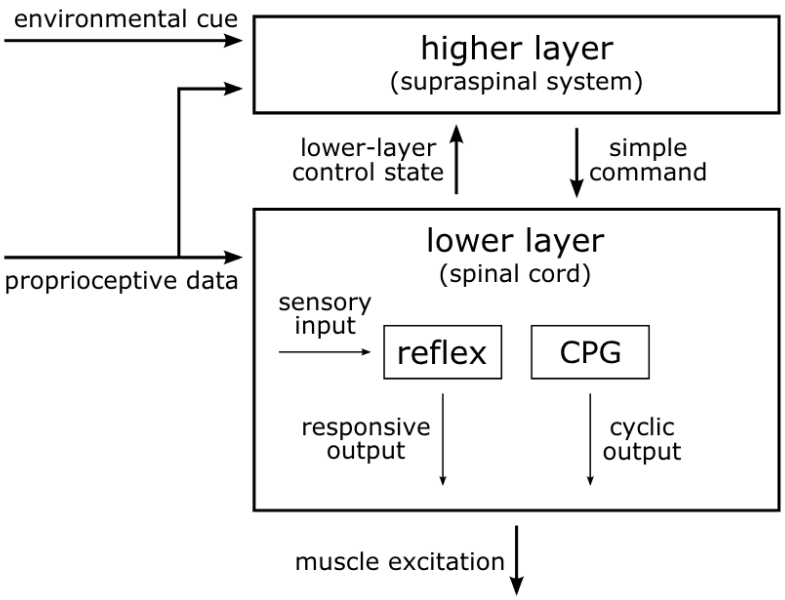

Figure 2. Scheme of human walking hierarchy in [2].

It was noticed that transfer from 2D to 3D space dramatically increases the solution's complexity. The authors prepared a simulation environment for competitors, which performs simulation and calculates a predefined reward function: the environment documentation [4] and the environment source code [6]. It is based on OpenSim [5]. The geometrical parameters of the model and its mass are close to the average human. The reward function is calculated at each step based on aliveness (not failing down), making steps, and following the velocity vector. Each new simulation step returns the current body state, including sensors and the desired velocity vector. The design scheme is presented in **Figure 3**.

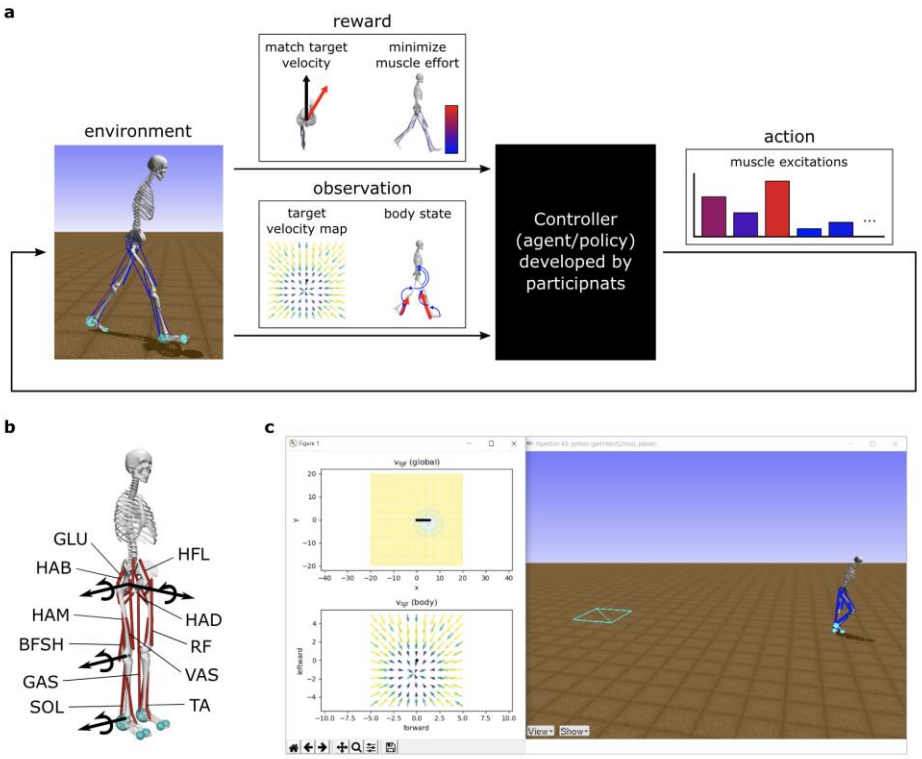

Figure 3. Design of simulation environment in [2]. a) Environment process flow. b) Musculoskeletal model used. c) Visualization.

The Reinforcement Learning approaches described in the paper can now be considered “state-of-the-art” techniques, at least to the authors' knowledge. They produce results that have not been seen before and include Curriculum Learning, Off-Policy Deep Reinforcement Learning Algorithms, DDPG, TD3, SAC, RAVE (Risk Averse Value Expansion), and others. The article provides a more detailed description of those techniques and related references.

Also, some participants used effective heuristic techniques to constrain possible solution space in the initial training stages: discretization of the action space, frame skipping, and reward shaping.

One of the ideas in this paper states that forces and other dynamic parameters of the model are much easier to measure during simulation than in real life for real humans. This potentially allows for simplifying and speeding up the development and testing of orthopedic devices like orthoses and prostheses or testing hypotheses about therapeutic exercises, which correlates with the objectives of my project.

The review of successful competitors shows that the environment provided is suitable for training a human model to walk. As it is also available as open source, I can reuse and modify it to suit my project's needs.

Limitations of this research, in the context of my project, include:

1. Strict focus on Reinforcement Learning, with quite a short overview of Evolutionary Algorithms.
2. Though evolutionary algorithms are mentioned, they are used mostly to find a good design for neural networks. One of the competitor teams implemented distributed computations based on an approach similar to populations in Genetic Algorithms, but that is the only significant mention in the paper.
3. Even the top solutions from the competitions were prone to getting stuck in a local maximum. My project will try to improve that flaw.

Parts of the paper, that are useful for my project, include:

1. Implementation of the simulation environment and reward function. That is a good start for my project, though they will be adjusted during implementation.
2. The idea of a hierarchical representation of walking allows for the better initial structuring of my project's design.
3. The heuristic techniques mentioned above are also quite useful for the initial stages of Genetic Algorithms, especially discretization of the action space and reward shaping.
4. There is a solid review of useful literature close to my project topic.

2.3. Learning To Run

[3] describes the top 8 submissions for the “NIPS 2017: Learning To Run” challenge. The competition was dedicated to learning a human biomechanical model to run through obstacles. All the top teams used Deep Reinforcement Learning, but some also attempted to use Evolution Strategies. The simulation environment is similar to the one described in the previous literature review.

The techniques used by competitors can be classified as “state-of-the-art” techniques and include DDPG (Deep Deterministic Policy Gradient) with different significant improvements, ACE (Actor-Critic Ensemble), PPO (Proximal Policy Optimization), TRPO (Trust Region Policy Optimization), and many others.

Though the paper is focused on Reinforcement Learning, it still provides valuable insights for my project. It shows the feasibility of the idea and describes many heuristic techniques that can easily improve the performance of Genetic Algorithms.

Issues mentioned in the paper my project aims to resolve include:

1. Making a model to bend knees is a problem in many submissions due to stacking in local minimums. As my project uses Genetic Algorithms, it should hypothetically be less prone to such issues.
2. The submissions are strictly focused on Reinforcement Learning. One of the teams tried to explore Evolution Strategies, and it performed better than DDPG but worse than PPO. It is not specified which Evolution Strategies were used, but they stated that "*ES usage in the Learning to Run environment should be more thoroughly examined*" [3].

Techniques and ideas from this paper that are expected to help to build my project include:

1. Reward Shaping. All the submissions experimented extensively with modifying the reward function to achieve better results: a penalty for straight legs, an angle of the head-pelvis line, and many others. I will use these techniques to tune my fitness function.
2. Frame Skipping/Action Repeat. Many of the submissions changed muscle activations only every 4-5 simulation steps instead of each step, significantly speeding up the convergence of their neural networks. I will also try this technique for my genetic algorithm to reduce the complexity of the patterns to be found.
3. Discretization of the action space is useful for constraining the exploration space and potentially may also decrease the complexity of my genetic algorithm.
4. Human walking has bilateral body symmetry adjusted for the phase difference between left and right leg muscle activation. This allows for a twofold decrease in the number of model inputs that should be calculated, which also may decrease the complexity of my Genetic Algorithm.
5. One of the teams recompiled OpenSim binaries with lower accuracy, providing 3x speedup in running simulations: [11]. My project may need to use these techniques to run more generations for the same amount of time.
6. Muscle activation inputs are values in the range [0, 1]. To constrain the exploration space, one of the teams considered muscle activations as binary inputs: 0 or 1. They iterated through different combinations to find combinations with the biggest reward. It may be worth trying to use a similar technique for building the first generation instead of a fully random generation.

2.4. Learning To Walk With Genetic Algorithm

[7] uses the environment from the "NIPS 2017: Learning To Run" challenge to teach a human model to walk directly in 2D space with Genetic Algorithms. **Figure 4** shows an example of the result achieved.

The source is an online article but still can be considered credible because:

1. It is published on "Towards Data Science", a resource that hosts many other useful articles that prove their credibility and performs at least a high-level review of the articles.
2. It provides source code, which I tested and successfully reproduced the results.

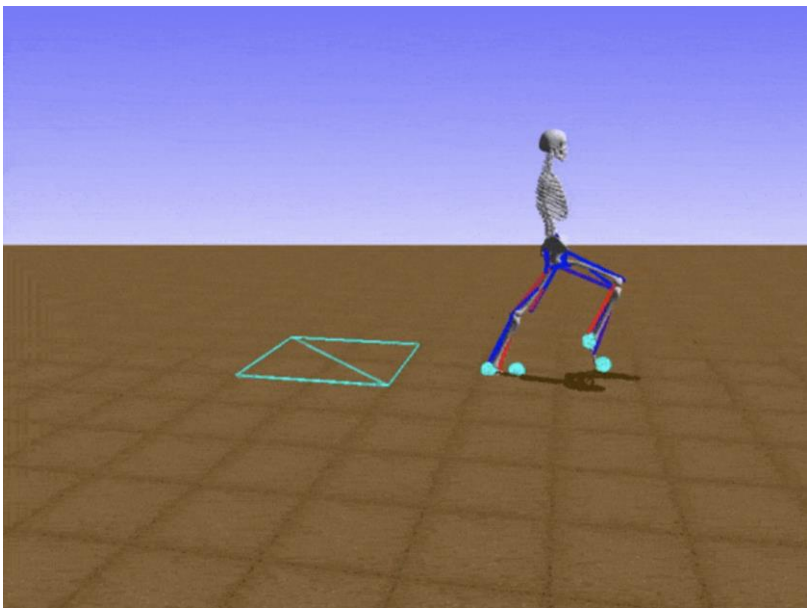

Figure 4. Example of the achieved walking in [7].

Though there are no “state-of-the-art” techniques used, the article introduces some important ideas in the context of my project:

1. Walking is a periodic movement, and the Fourie Transform can be used to present an oscillation of a single muscle during a period. That potentially allows for decreasing the size of the genome.
2. The idea of my project is feasible, at least in 2D space.
3. It provides the source code, which I can use as a starting point for my project.

Parts that my project will improve:

1. The article's Genetic Algorithm is more similar to Random Search. It does not include population and recombination, only regular mutations of a single instance. My project will implement a full-fledged Genetic Algorithm.
2. The article's Fourie Transform results are not normalized to the range [0, 1], leading to blinking inconsistencies between the genotype and phenotype.
3. Only 4 first cosine and phase components of the Fourie Transform are used. My project will use a fuller version of the transform.
4. 2D space is used, while my project will also use a 3D simulation environment.

2.5. Physiology of human walking

The authors of [9] use surgically implanted sensors, mechanical calculations, and musculoskeletal modeling to measure and analyze forces in the lower limb muscles of ten people during walking. They found that the muscles intended for supporting and propulsion of the body generate greater and more persistent forces while the muscles that create moments, collinear or orthogonal to the ground reaction force, spend less energy and are more changeable. The resulting muscle force curves are shown in **Figure 5**.

The article claims that the used approaches should be recognized as “state-of-the-art” at the moment of publication because they combine several methods, some of which (sensor implants) are at the peak of technology.

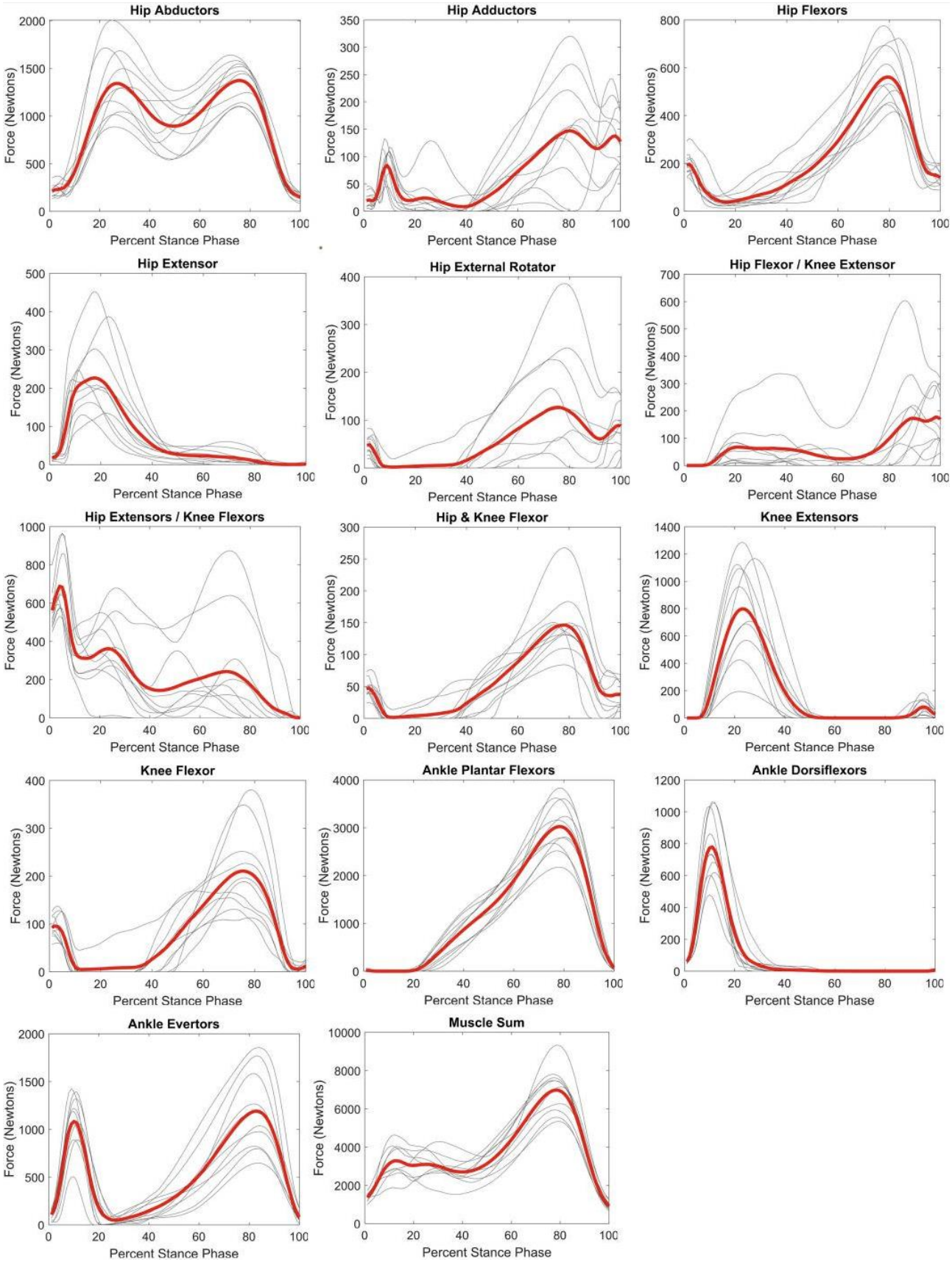

Figure 5. Force profiles of lower limb muscles during walking from [9]. The grey color shows individual curves, and the red is the averaged curve.

In the context of my project, those forces may be considered tension curves, which correspond to the activations of muscles in the model used. I will use them as references for a general look at how initial randomly generated activations should look and for comparison with resulting activations plots.

2.6. Reinforcement Learning Versus Evolution Strategies

Genetic Algorithms may be considered as a subfamily of Evolution Strategies. Some people even consider them as synonyms. [10] provides a high-level description of RL and ES algorithm families and compares them in different aspects separately and as a hybrid approach without sticking to specific algorithms. See **Figure 6** for the article structure.

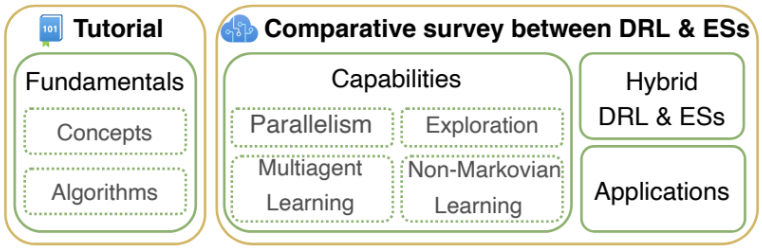

Figure 6. Structure of the survey [10].

See **Figure 7** for the design schemes of RL, DRL, and ESs.

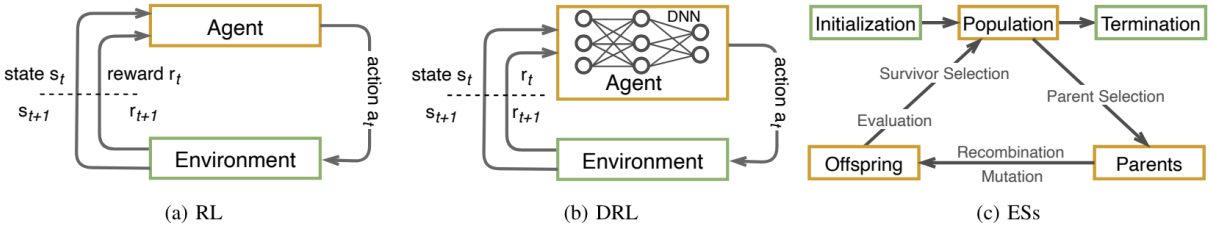

Figure 7. Schematic difference between RL, DRL, and ESs in [10].

The article makes its conclusions based on “state-of-the-art” approaches from both RL and ESs.

Main statements related to my work include:

1. Parallelism accelerates the execution of both DRL and ES algorithms. So, my project will implement a parallelized Genetic Algorithm.
2. The exploration-exploitation dilemma, or, in my case, a balance between preserving the best ones from each population and mutability, does not have a universal solution and often requires sophisticated and creative approaches.
3. Evolution Strategies algorithms are less prone to getting stuck in local optima than DRL algorithms. This is one of my project's arguments for trying a Genetic Algorithm.
4. Both DRL and ES algorithms may be applied in robotics, games, finance, and many other fields, which supports my project's objectives.
5. ESs are less popular than DRL, perhaps because, subjectively, DRL may seem to have richer capabilities in the final result and in the research. My project aims to show that Genetic Algorithms are not less useful.

3. Design

3.1. Project Overview

This project uses a genetic algorithm to train a biomechanical human model to walk in the simulation environment.

I use OpenSim virtual environment [5] and Python library [15] for simulation management. Human models *gait14dof22musc_planar_20170320.osim* and *gait14dof22musc_20170320.osim* (2d.osim and 3d.osim in the project files respectively) are reused from [6]. The environment for both models is a 3D space with a 2D plane used as a floor. The 2D model is constrained to not make any movements and to not fall to the left or the right. They both look like skeletons with muscles, initially placed in the plane's center. See **Figure 7** for a visual demo and axes in the space.

The model has 22 muscles in total, 11 per leg. Each muscle accepts a real number from 0 to 1 as an input, which dictates muscle tension in each simulation step. Each simulation runs for 1000 steps or until the model falls. A single step is considered to be 0.01 seconds.

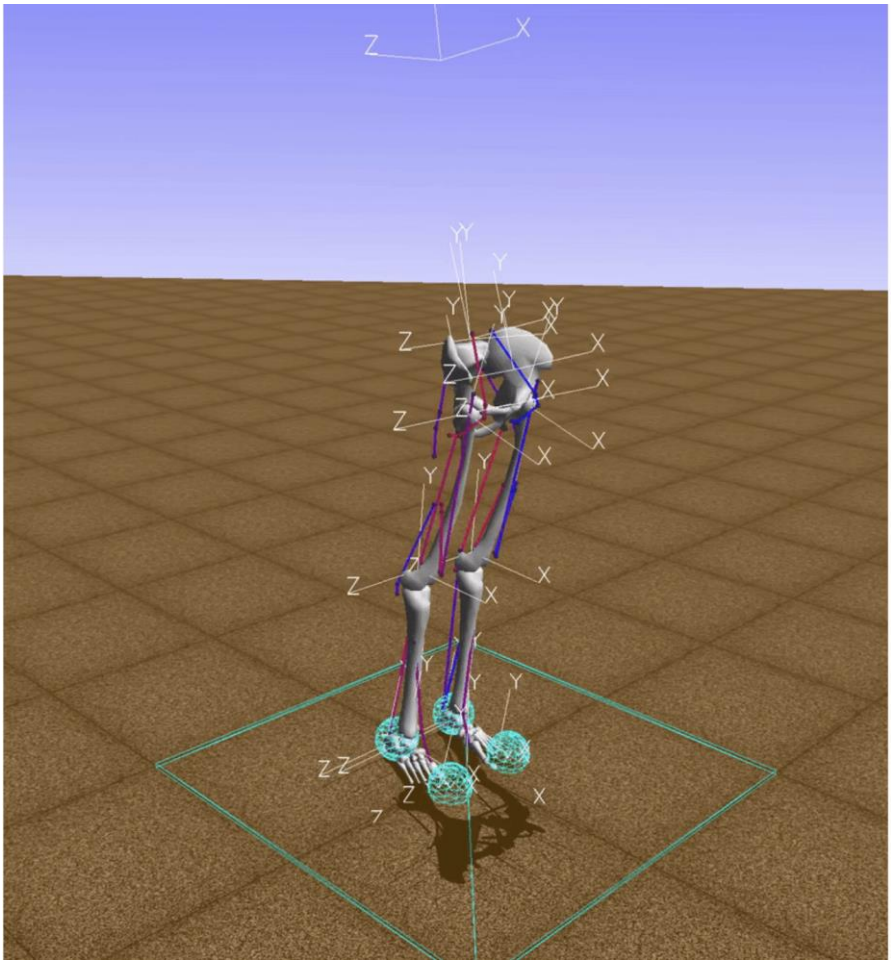

Figure 8. The human model in the OpenSim environment with axes of direction.

As human gait is a periodic process, I encode activations as a periodic function for each muscle. The details of exploration space are described in the genotype description below in 3.3. The overall structure of the project.

3.2. Domain and users

The project can be related to several groups of domains, each of which includes its own end users.

3.2.1. Artificial Intelligence, Evolutionary Algorithms, Machine Learning, Automatic System Design

This project implements a Genetic Algorithm, a subfield, or, in many contexts, a synonym of Evolutionary Algorithms. All of them are related to methods of implementing Artificial Intelligence. According to [3] and [10], Genetic Algorithms often receive less attention than Deep Reinforcement Learning. My work may provide a better overview and an effective demonstration of Evolutionary Strategies and, particularly, Genetic Algorithms for DL practitioners and other AI engineers, to make them consider GA as an option worthy of note.

3.2.2. Evolutionary Biology, Anthropology

Different types of neuromusculoskeletal human models are used in biology and anthropology to explain evolution and different aspects of human development during history and in modern times. Biologists, evolutionary biologists, and anthropologists may use this project to optimize their research and gain a deeper understanding of how the human gait works and which evolutionary stages might lead to acquiring such a skill.

3.2.3. Rehabilitation Medicine, Orthopedy

Rehabilitologists and orthopedists often use ready-made exercises and devices or develop new ones to improve, recover, or imitate human movement, including its power, speed, and dynamic range. Finding the optimal movement strategy or device configuration is a nontrivial task, which can be made easier if the results of this project are used to perform experiments in simulation with a walking model instead of real humans.

3.3. The overall structure of the project

The architecture is schematically described in **Figure 9**. The block scheme of the genetic algorithm and Fourier transform are shown and described in 3.5. Methods.

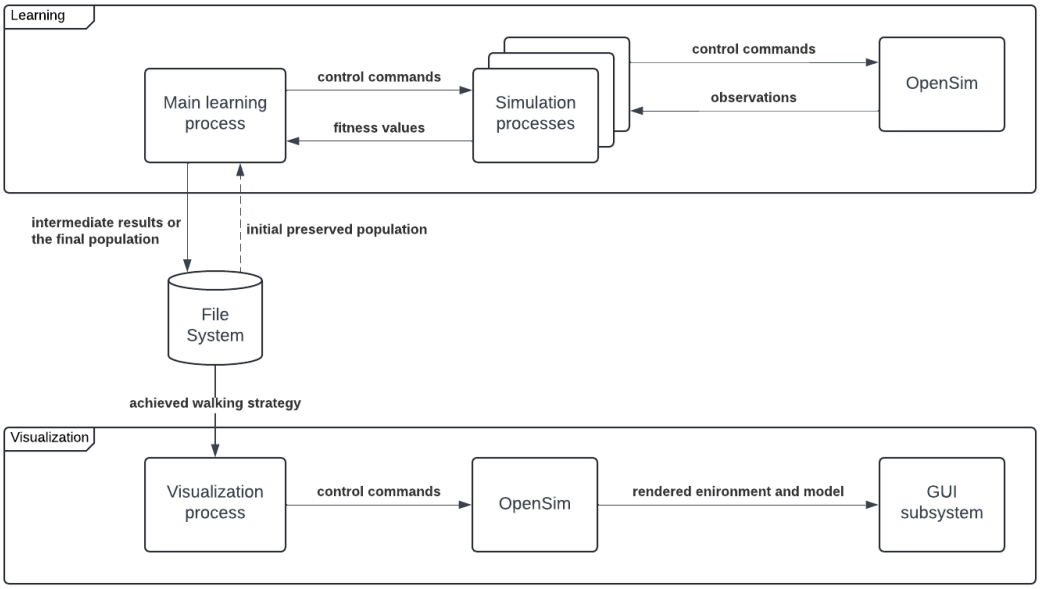

Figure 9. Scheme of project architecture.

Each instance of a walking strategy has 11 muscles per leg. **Figures 10-20** show screenshots of those muscles on the right side of the human model from OpenSim UI, along with their codes and names. The same muscles are duplicated on the left side.

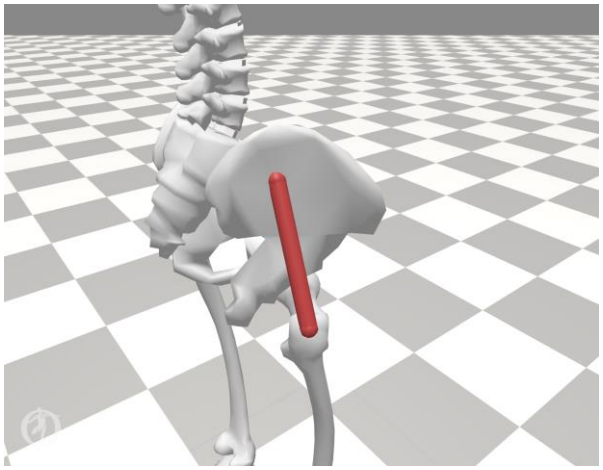

Figure 10. HAB: hip abductor.

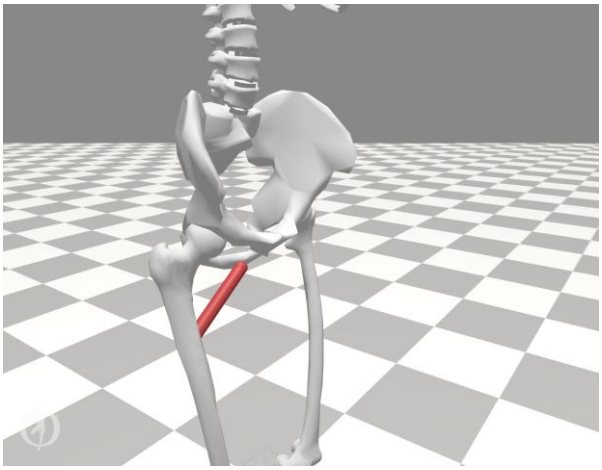

Figure 11. HAD: hip adductor.

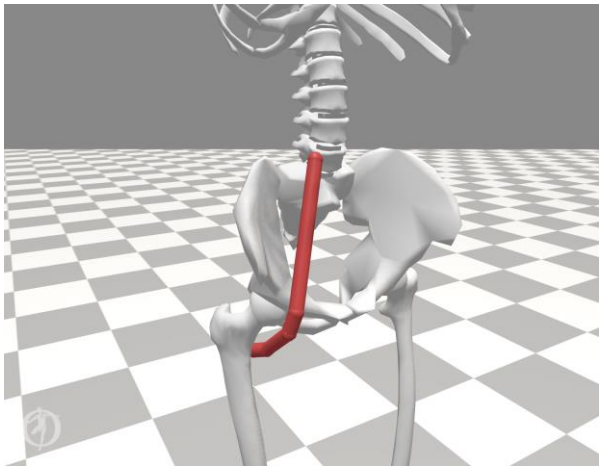

Figure 12. HFL: hip flexor.

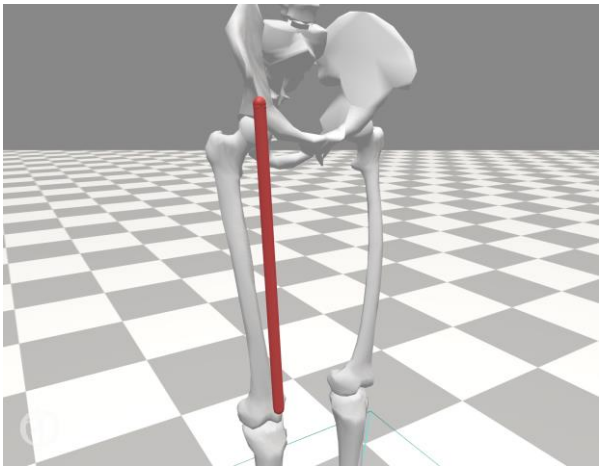

Figure 13. RF: rectus femoris (biarticular hip flexor and knee extensor.

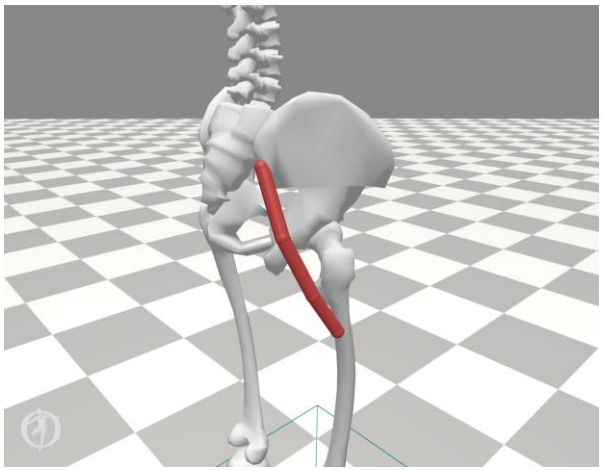

Figure 14. GLU: glutei (hip extensor).

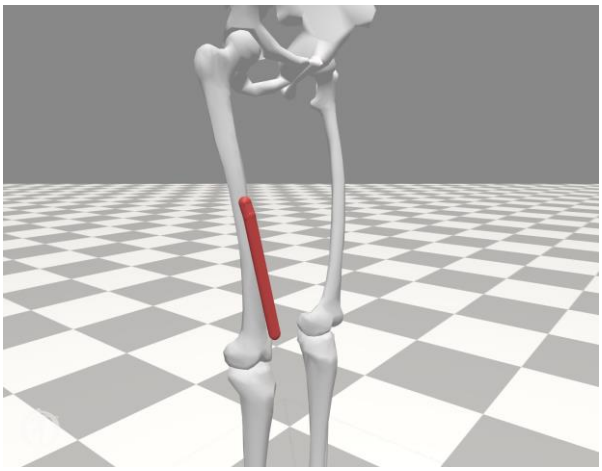

Figure 15. VAS: vastii (knee extensor).

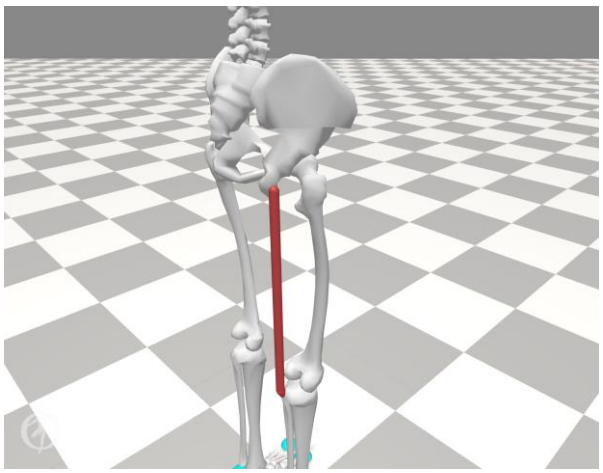

Figure 16. HAM: hamstrings (biarticular hip extensor and knee flexor).

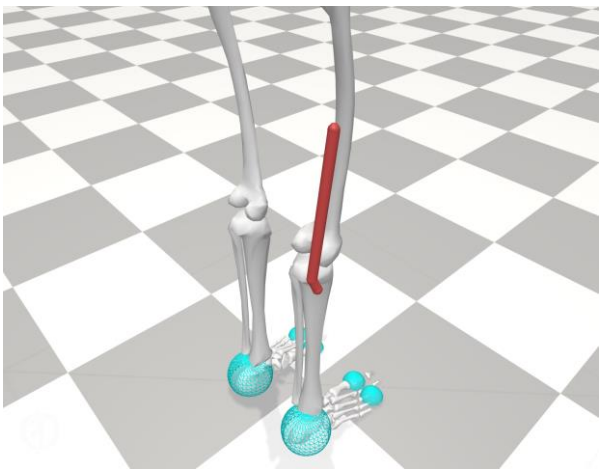

Figure 17. BFSH: biceps femoris, short head (knee flexor)

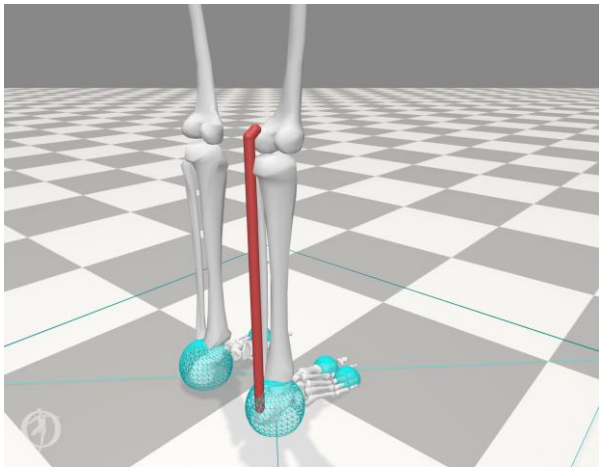

Figure 18. GAS: gastrocnemius (biarticular knee flexor and ankle extensor).

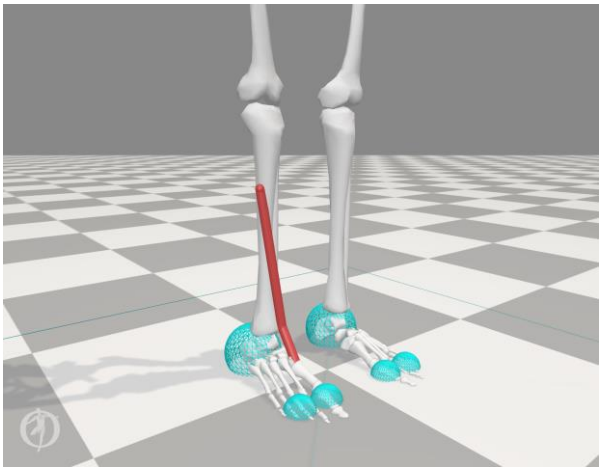

Figure 19. TA: tibialis anterior (ankle flexor).

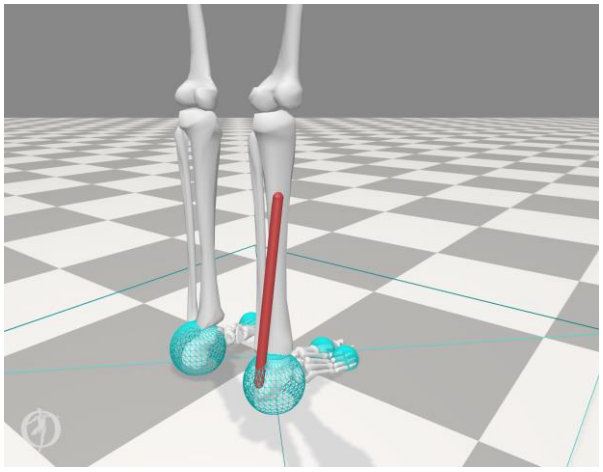

Figure 20. SOL: soleus (ankle extensor).

The encoding scheme represents a genotype as metadata and a 2D array of direct activations or Fourier coefficients, and a phenotype as a muscle activation function built from the values of those arrays. See **Table 1** for details of a muscle's genotype.

Genotype property	Description	Possible Values
Type	Type of representation of the activation function values: direct values, or coefficients of its Fourier Transform.	direct | fourier
Period	Duration of a single walking period in simulation steps.	120 | 160 | 200 | 240 | 320 | 400
Sampling interval	Duration of a sampling interval in simulation steps. A sampling interval is an interval that has constant values of the activation function to reduce the resolution of the function and simplify its training.	5 | 10 | 20 | 40
Precision	How many values of the function representation to keep. In the case of Fourier representation, it states how many first Fourier coefficients to use (I assume that the first coefficients are the most significant). In the case of direct values, it means how many uniformly distributed direct values will be actually calculated, while gaps will be filled with interpolation.	5 | 10 | 20
Components	The values of the function representation.	<precision> of real or complex values

Table 1. Structure of genotype in the encoding scheme.

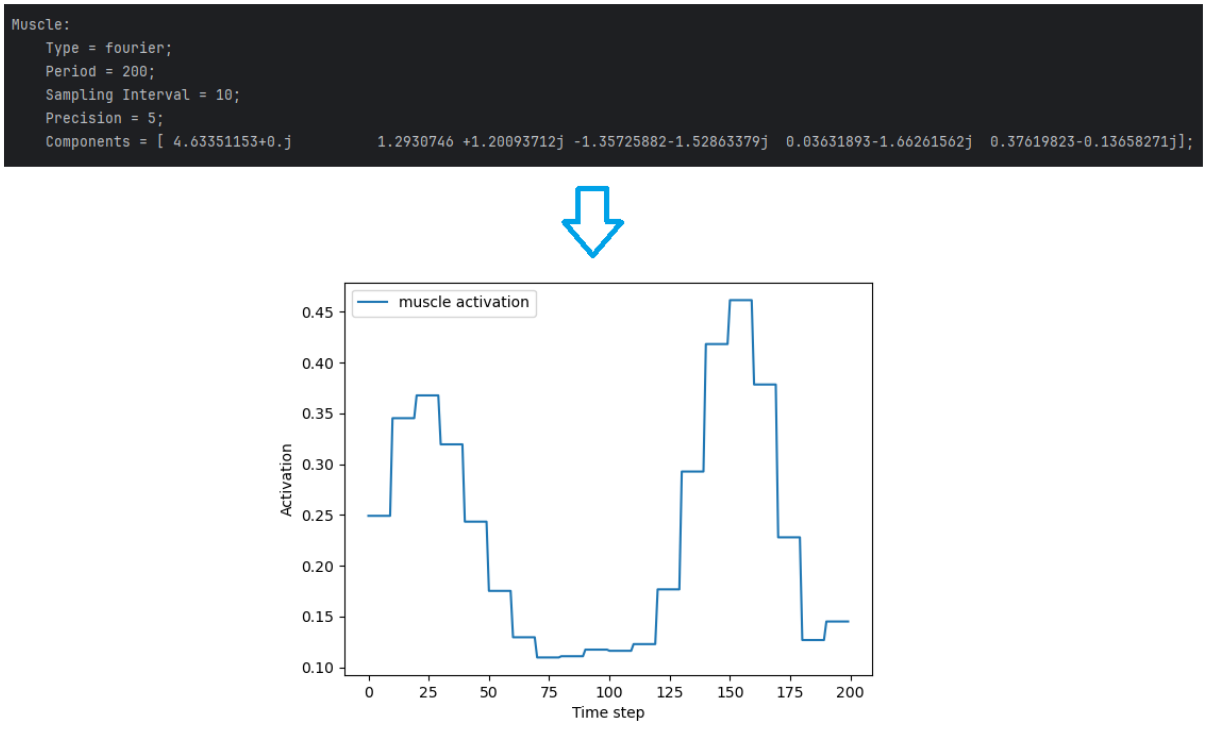

Figure 21. Example of genotype → phenotype transformation.

Figure 21 shows an example of a random genotype and resulting phenotype.

Fourier Transform is chosen as one of the options because, unlike straightforward activation values, it allows the management of basic frequency components of the function, resulting in smoother functions and more meaningful changes while mutating.

The fitness function is calculated using **Equation (1)**, where:

- N - number of simulation steps.
- i - current simulation step.
- r_l - reward for staying alive, always *0.1*.
- d_{ix} - delta of pelvis position for the last time step across x-axis.
- d_{ixz} - delta of pelvis position for the last step on the xz plane.
- p_{ci} - penalty for crossing legs. It equals 1, if legs are crossed, and 0 otherwise.
- r_{fi} - reward for a single footstep. It equals 0, if there is no footstep finished on the current timestep, and **Equation (2)** otherwise, where:
  - i - current simulation step, when the footstep is finished.
  - i' - number of the simulation step, when the previous footstep was finished, or when the simulation started, if there were no footsteps yet.
  - j - simulation step between i' and i
  - t_s - time duration of a single time step, a constant value equal to 0.01.
  - k - index number of a muscle.
  - a_{kj} - activation of muscle k during a step j, [0, 1].

$$fit = \sum_{i=1}^{N}(r_l + 10 \cdot d_{ix} \cdot \frac{|d_{ix}|}{|d_{ixz}|} - p_{ci} + r_{fi}) \quad (1)$$

$$r_{fi} = 20 + 10 \cdot \sum_{j=i'}^{i} t_s - \sum_{j=i'}^{i}\sum_{k=1}^{22}(t_s \cdot a_{kj}^2) \quad (2)$$

Figure 22 shows the crossover scheme. *switch_index* is defined randomly during each crossover. Each circle means a muscle in a walking strategy. Parent walking strategies are chosen using a *roulette wheel selection* based on their fitness values.

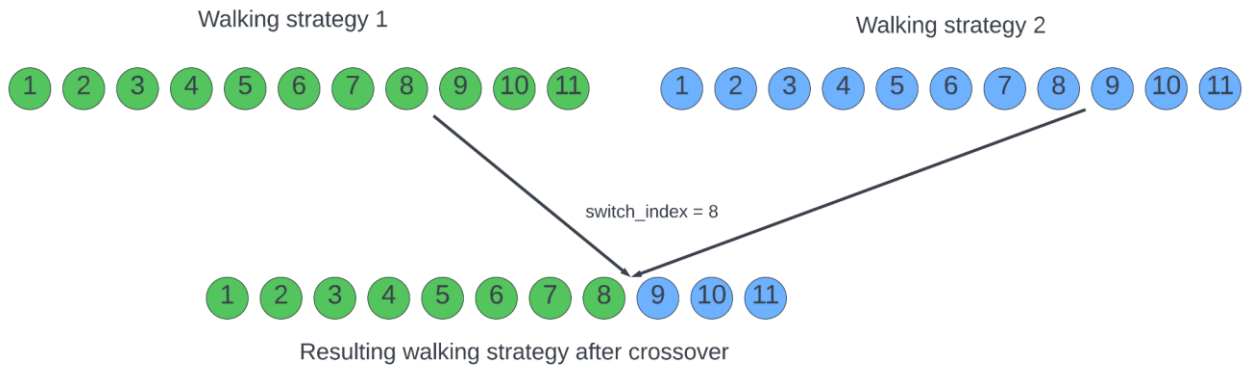

Figure 22. Crossover scheme.

Mutation is defined as randomly choosing from available values for type, period, sampling interval, and precision. It is an addition of normally distributed values for components, so smaller changes are more probable than bigger ones. Mutation of components has a higher probability than mutation of the metadata.

3.4. Technologies

The genetic algorithm is to be run on a Windows PC with an AMD Ryzen 7950X CPU, which has 16 cores, 32 logical processors, and a clock frequency of 4.5 GHz.
The main technologies used include:

1. Python [16].
2. OpenSim [5].
3. NumPy [17] for numerical calculations.
4. perlin_noise [18] for generating Perlin noise.
5. SciPy [19] for quadratic interpolation.

3.5. Methods

3.5.1. Genetic Algorithm

Figure 24 presents the algorithm as a block scheme, where

- *POPULATION_SIZE* is the number of walking strategies in a single generation.
- *MUTABILITY_INCREASE_THRESHOLD* is the number of generations without improvement, after which mutation rates are increased.
- *MUTABILITY_DECREASE_THRESHOLD* is a number of generations with improvement, after which mutation rates are decreased.
- *ELITES_NUMBER* is a number of the best walking strategies in each generation to preserve for the next generation.

3.5.2. Fourier Transform

The tension of a single muscle is a discrete function of the time step, which can be represented as a Fourier Series using **Equation (3)**, where

- n - time step.
- i - sequential number of a Fourier Series component, or a harmonic number.
- c_0 - real number and constant Fourier term which represents an average value of the function during the period.
- c_i - *i-th* Fourier term, a complex number that represents the variable part of the function.
- T - period measured in time steps.

$$f(n) = \frac{c_0 + \sum_{i=1}^{T} (\Re(c_i) \cos(\frac{i \cdot 2\pi \cdot n}{T}) - \Im(c_i) \sin(\frac{i \cdot 2\pi \cdot n}{T}))}{T} \quad (3)$$

In this project, I will use ready FFT and IFFT implementations from NumPy to optimize calculation speed.

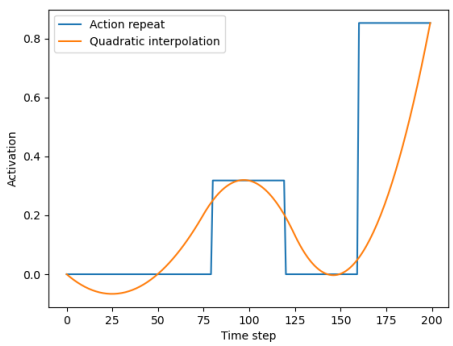

Figure 23. Example of Action Repeat and Quadratic Interpolation for the same activation function.

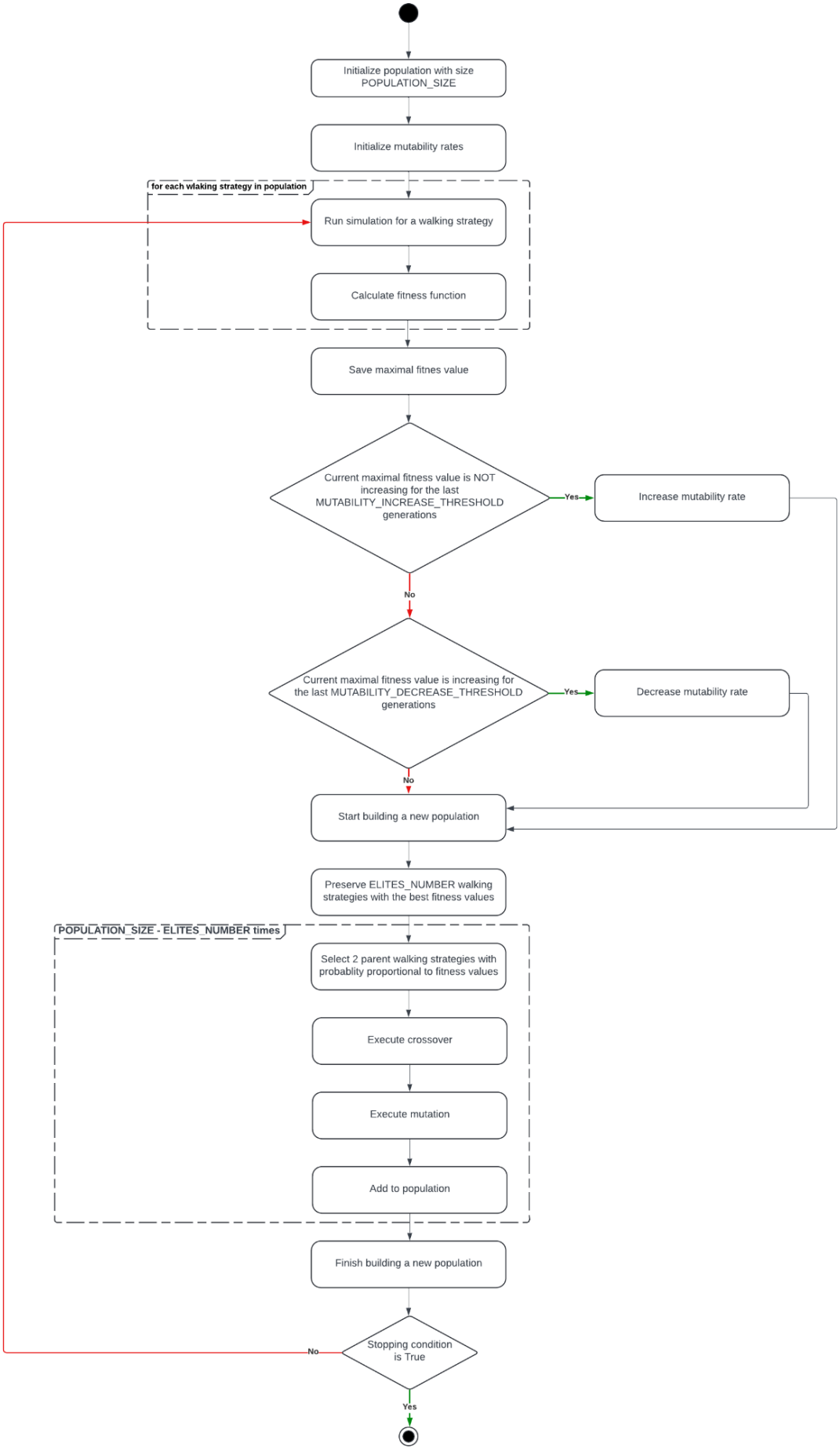

Figure 24. Block scheme of the genetic algorithm.

3.5.2. Action Repeat and Quadratic Interpolation

This project will use the Frame Skipping technique, where activation values will be calculated once per some number of time steps (sampling interval) instead of calculation in each step. I will consider two approaches about how to fill activation values in a single sampling interval:

1. Action Repeat. The same value is just repeated during the sampling interval.
2. Quadratic Interpolation. The values are interpolated between the value of the current sample and the value of the next sample using quadratic polynomial interpolation. I consider quadratic interpolation specifically because it balances smoothness between linear interpolation with too ragged result, and cubic interpolation with more added noise.

See **Figure 23** for examples of both techniques.

3.6. Testing and evaluation plan

I will test and evaluate the project against two groups of criteria. Technical criteria are important to avoid performance flaws and mistakes in basic algorithms. Functional criteria ensure that the final result meets the initial objectives and the degree of this compliance.

3.6.1. Technical criteria

1. **Unit test coverage.** Measured in the percent of lines of the source code executed during running the unit tests. The plan is to cover at least the code with the most sophisticated logic, like the encoding scheme, crossover, mutation, and breeding. The expected rate is not less than 50%.
2. **Duration of simulation of a single population.** Measured in seconds. This is the total time spent on running all the walking strategies. The expected duration is not more than 1 minute per population of 150 walking strategies in a multi-threaded environment. It can be increased proportionally to population size and in the latest generation, when simulation may last longer than initially.
3. **Duration of evaluation and generation of a new population.** Measured in seconds. This is the time spent on calculations between running simulations and includes preparing fitness values, selecting parents, crossover, and mutation. The expected duration is not more than 5 seconds, depending on the encoding scheme and mutability rates.

3.6.2. Functional criteria

1. **Walking stability.** Number of simulation steps, the model is alive (does not fall).
2. **Distance.** The distance on the *x-axis*, the model traveled until falling or the end of the simulation.
3. **Energy consumed per 1 meter of the distance traveled.** See **Equation (4)** where
   - E - total energy consumed.
   - N - number of simulation steps.
   - i - current simulation step.
   - j - index number of a muscle.
   - 0.01 - normalization coefficient per a single simulation step.
   - a_{ij} - activation of muscle j during a step i, [0, 1].
   - d - total distance traveled, in meters.

$$E = \frac{\sum_{i=1}^{N} \sum_{j=1}^{22} (0.01 \cdot a_{ij}^2)}{d} \qquad (4)$$

4. **Subjective visual comparison of gait with top solutions from NIPS "Learn to Move" challenges.** [12, 13, 14] can be considered “state-of-the-art” solutions for more global

problems but still provide a reasonable visual demonstration of human walking. While [13] and [14] are more focused on running, [12] also shows different walking styles. I will evaluate how walking evolved in my project, comparing it to walking in the referenced videos.

5. **Learning rate.** Measured in an average increase of the maximal fitness value from a population per single generation. Also, the plot with the best fitness value per generation will be built.

4. Implementation

The source code is available at https://github.com/bastrich/ga-walking.

4.1. Initial population

The first task is to generate an initial population with random walking strategies and muscles. When generating with total random, I mostly have activation functions as shown in **Figure 25** which looks too ragged to be a muscle tension function, especially if refer to **Figure 5**, which shows real examples.

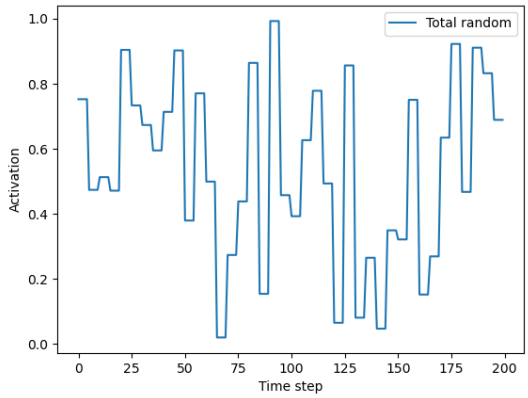

Figure 25. Activation function generated with total randomness.

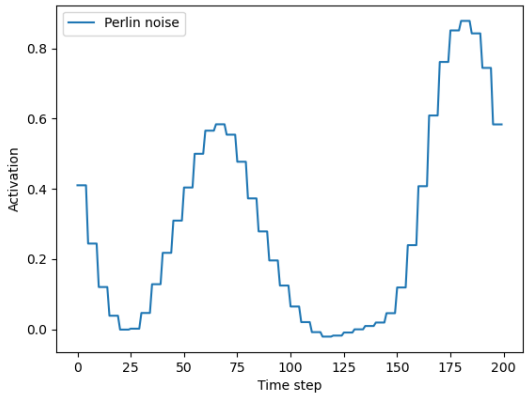

Figure 26. Activation function generated with Perlin noise.

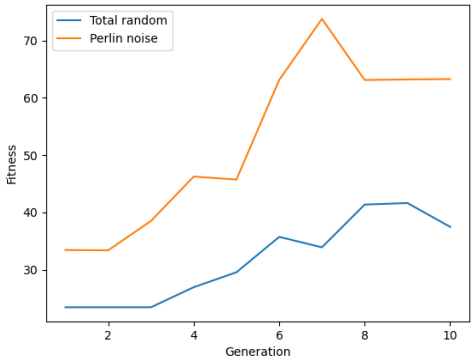

Figure 27. Comparison of fitness value of the 2D model for an initial generation with random and with Perlin noise.

However, when using Perlin noise, the resulting random function looks like in Figure 26, which is more similar to the real examples. **Figure 27** shows how the learning rate is improved in such case.

The source code for generating Perlin noise can be found in *walking_strategy.muscle.Muscle.generate_random_components.*

4.2. Encoding scheme

The encoding scheme for a 2D model is implemented according to 3. Design. **Figure 28** shows the distribution of genotype values among the population by generation.

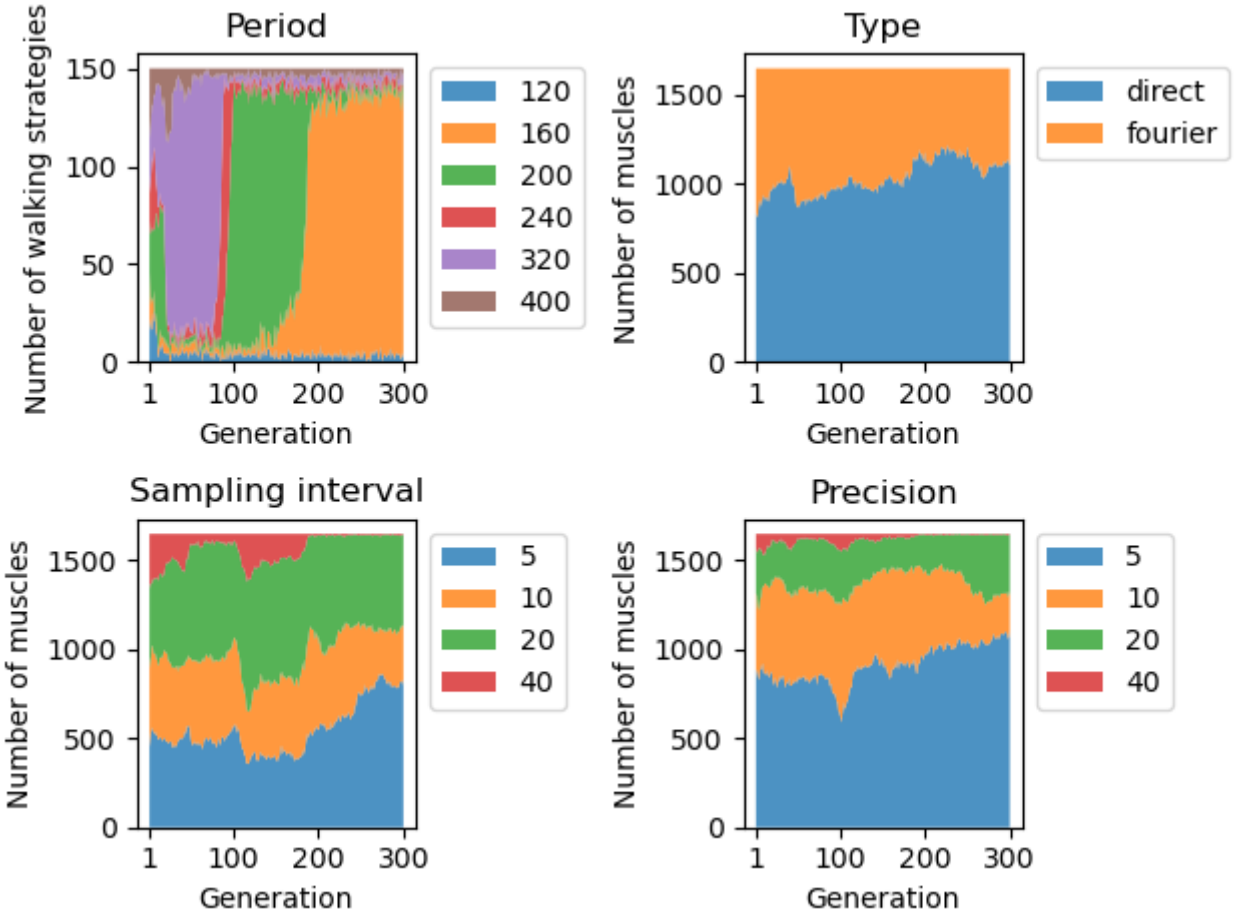

Figure 28. Distribution of genotype of 2D model across generations.

The 3D environment is too complex and requires more learning time to achieve comparable results, so, based on the genotype distribution for 2D, I simplified the encoding scheme for 3D: the period is always 160, the sampling interval can be 5 or 10, and the available precisions are 8, 16, and 32. **Figure 29** shows the genotype distribution for a 3D model.

Action repeat is chosen over Interpolation because it shows a little better performance and needs fewer computational resources. See **Figure 30** for a comparison.

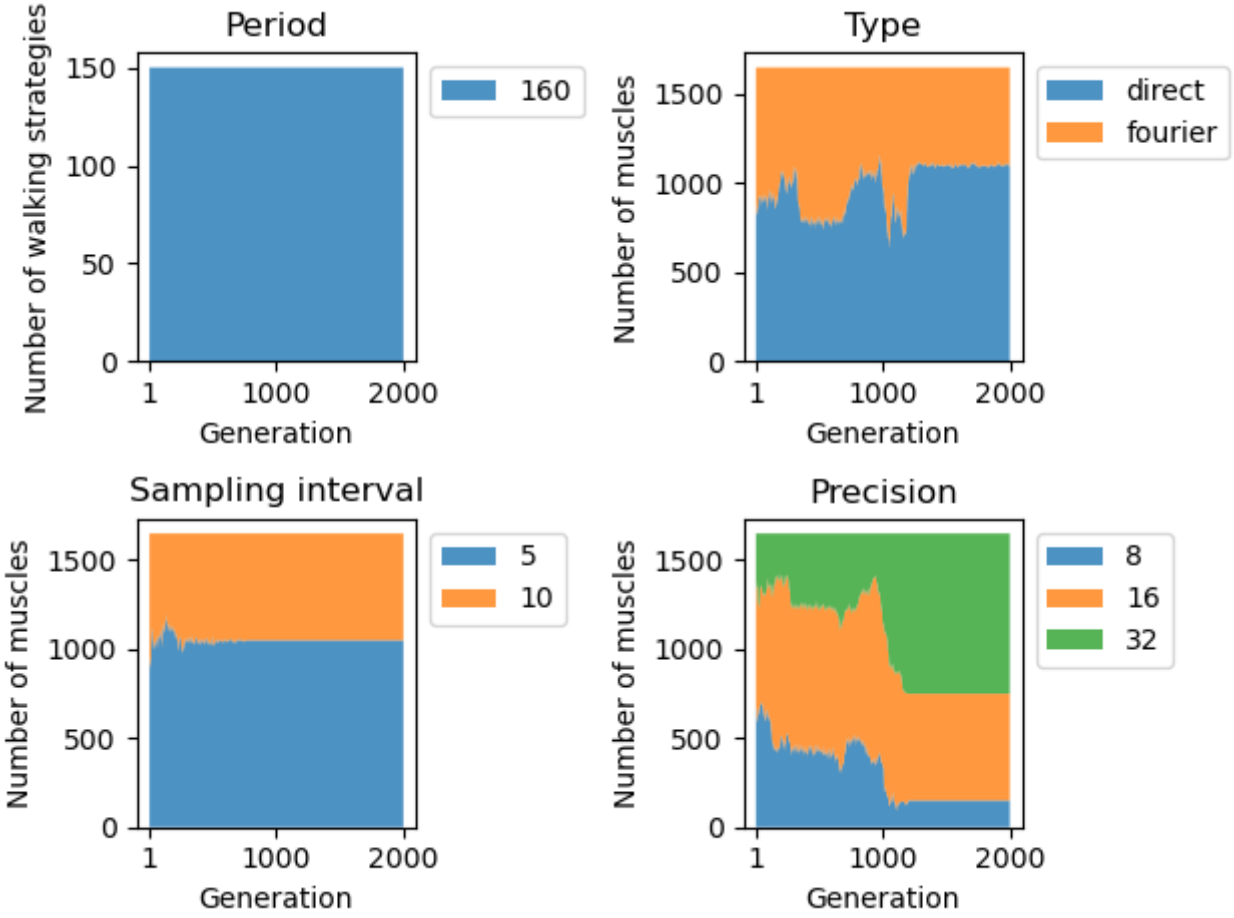

Figure 29. Distribution of genotype of 3D model across generations.

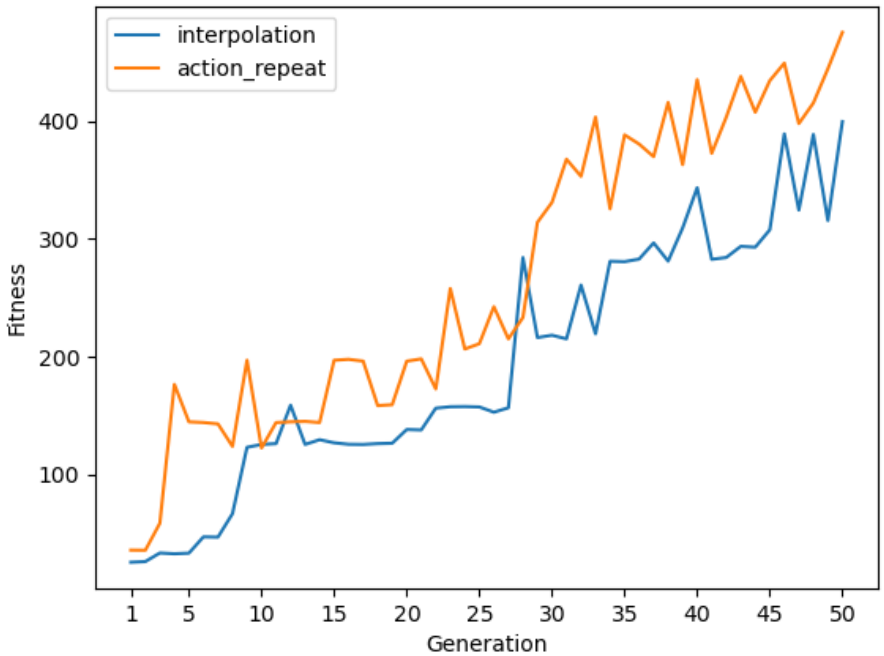

Figure 30. Comparison of fitness value when using Interpolation or Action repeat for 2D model.

Figure 31 shows changes in fitness values when using fixed genotype types or combining them. Though *direct* shows a better learning rate, the final fitness is the same, and **Figures 28, 29** demonstrate that *fourier* still has a significant presence in the final generation, so I decided to leave both types.

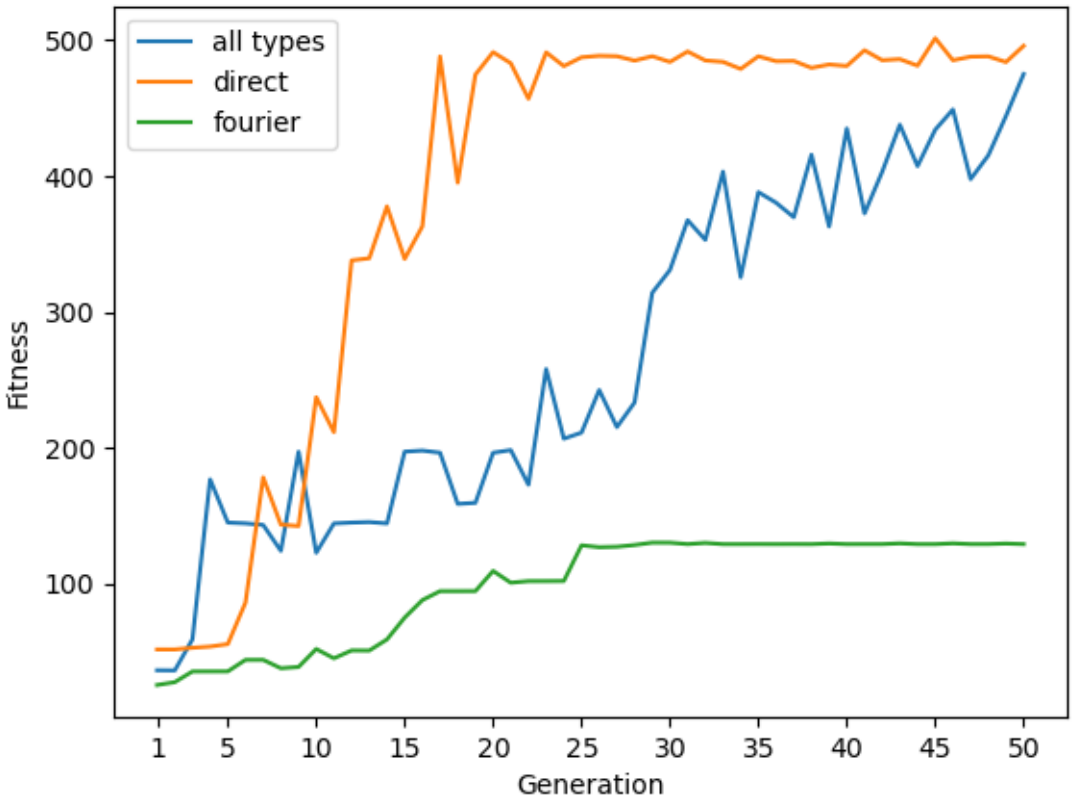

Figure 31. Comparison of fitness value when using different types of genotype.

Figure 32 shows screenshots of a 2D model during walking resulting from an implemented encoding scheme.

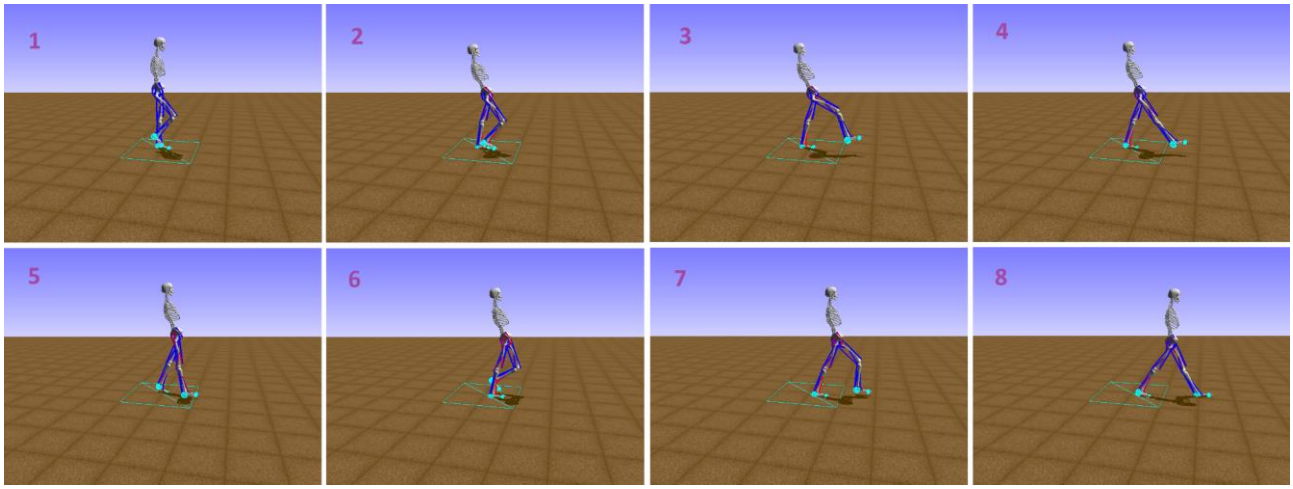

Figure 32. Screenshots of a visualized environment at intervals of 20 steps, starting from the first.

The source code of the encoding scheme can be found in *walking_strategy.muscle.Muscle*. The constructor verifies and initializes the genotype, and *calculate_activations* transforms the genotype into a phenotype.

4.3. Fitness function

The fitness function significantly impacts a learning rate. When it tracks only some basic metrics (for example, only distance and footsteps), allowing for many different development paths, or when it is, vice versa, too restricted (for example, considers only footsteps of a specific size in a specific direction with specific body angles), the solution is prone to stuck in local minima for many generations. It is important to find a balance between immediate and long-term rewards. I used visual verification as a testing method: if something did not look right, I changed the implementation of the fitness function and restarted the learning process.

If using a fitness function from 3. Design, the model quickly learns huge steps but fails after the 1st step and, thus, stacks near the starting point for a long time. To avoid that, I modified the fitness function so r_{fi} is replaced with **Equations (5, 6)**, where:

- T - period of walking in simulation steps.
- n_f - ordinal number of a footstep.

$$r_{fi} = 20 + 10 \cdot r_{fli} - \sum_{j=i'}^{i} \sum_{k=1}^{22} (t_s \cdot a_{kj}^2) \tag{5}$$

$$r_{fli} = \begin{cases} min(\sum_{j=i'}^{i} t_s, 0.8 \cdot \frac{T}{4} \cdot t_s), & \text{if } n_f = 1 \\ min(\sum_{j=i'}^{i} t_s, 0.8 \cdot \frac{T}{2} \cdot t_s), & \text{if } n_f > 1 \end{cases} \tag{6}$$

Such a modification continues rewarding the solution for bigger steps but sets the upper reward limit to *0.8* of a semi-period (as a period should have exactly 2 steps). This approach allows for setting the upper limit of a step without knowing its length and improves walking stability and distance as shown in **Figures 33, 34**.

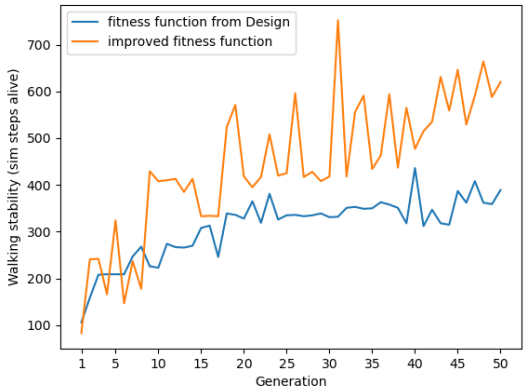

Figure 33. Dependency of walking stability (sim steps alive) on generation for different versions of a fitness function.

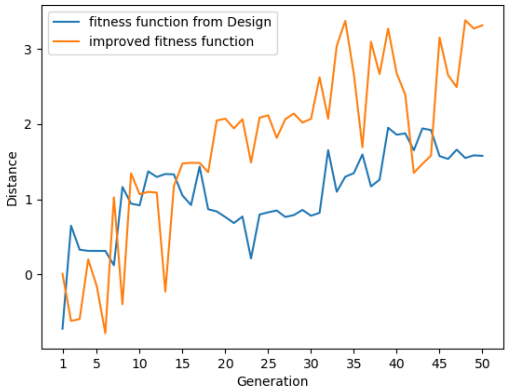

Figure 34. Dependency of distance traveled (sim steps alive) on generation for different versions of a fitness function.

The 3D environment required significant changes: simplify the formula but add more penalties for wrong movements. See **Equation (7)** for details, where:

- r_l - reward for staying alive. It equals *0.1*, if the current timestep is less than a period, and 0 otherwise.
- p_{ph}, p_{ps}, p_{hp}, p_{pr}, p_{pb} - penalties for pelvis height angles and rotations
- r_{fi} - reward for a single footstep. It equals *20*, if there is a footstep finished on the current timestep, and 0 otherwise.

$$fit = \sum_{i=1}^{N}(r_l - p_{ph} - p_{ps} - p_{hp} - p_{pr} - p_{pb} + 30 \cdot d_{ix} \cdot \frac{|d_{ix}|}{|d_{ixz}|} - p_{ci} + r_{fi}) \quad (7)$$

Without that modification, the learning rate was very low and I could not achieve 1-2 steps even after 2000 generations.

The source code of the fitness function can be found in *sim.sim.Sim.calculate_current_fitness.*

4.4. Genetic Algorithm

The genetic algorithm is implemented according to 3. Design. The population size is 150 walking strategies, as I found experimentally that such a number is enough for the progression of the genetic algorithm. It allows the execution of a simulation for the whole population within 1 minute, which is one of the evaluation criteria. The number of generations is 300 for 2D and 2000 for 3D. Such amounts were chosen as they allow for noticeable results.

Figures 35, 36 show muscle activation plots of developed walking strategies compared to initially generated generation, and **Figures 37, 38** show the progress of the fitness value. Those figures along with screenshots of the first step in **Figures 39, 40** demonstrate that the algorithm works as expected and the found solution improves during its execution.

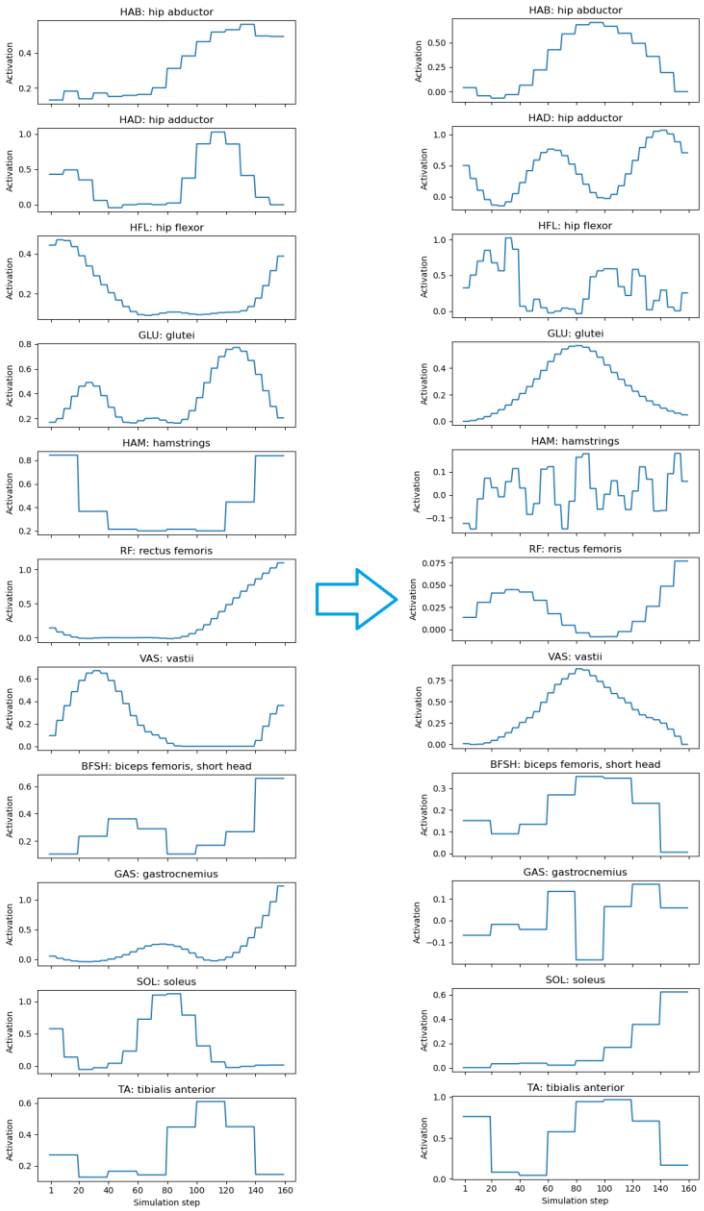

Figure 35. Muscle activations of 2D model: initial and after completion of the genetic algorithm.

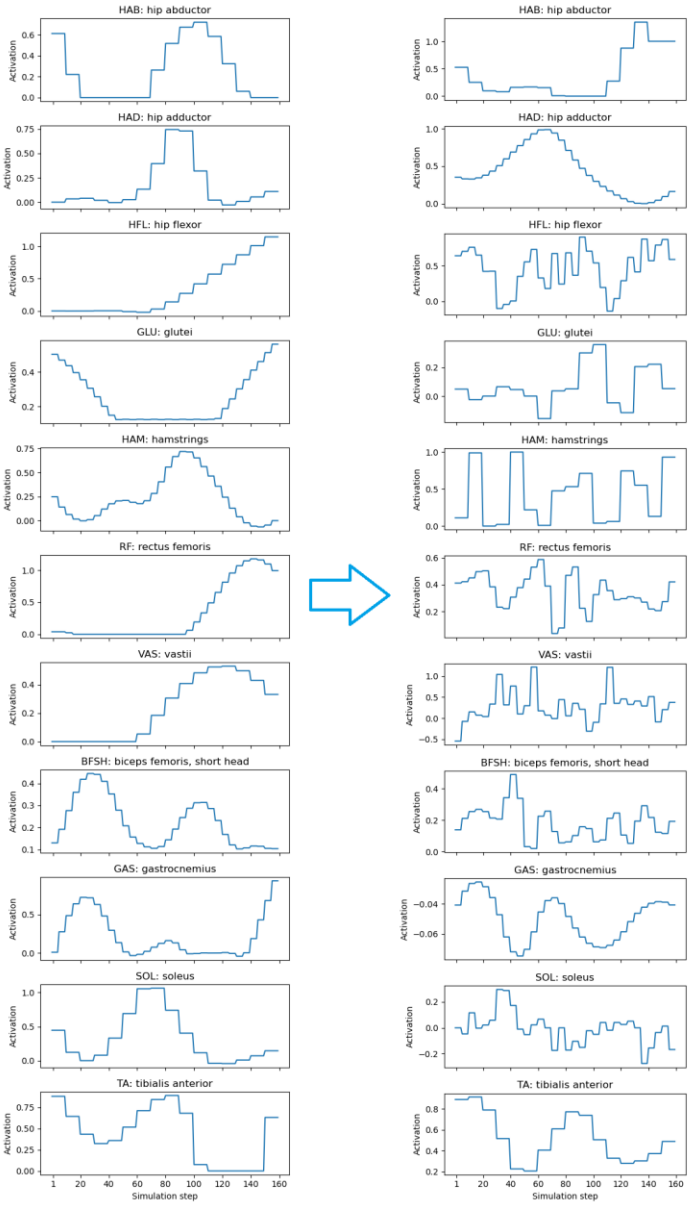

Figure 36. Muscle activations of 3D model: initial and after completion of the genetic algorithm.

In the context of this project, the encoding scheme, mutability approach, and fitness function are the most important parts of the implementation. Tweaking and adjusting them provided the most significant impact on the final results.

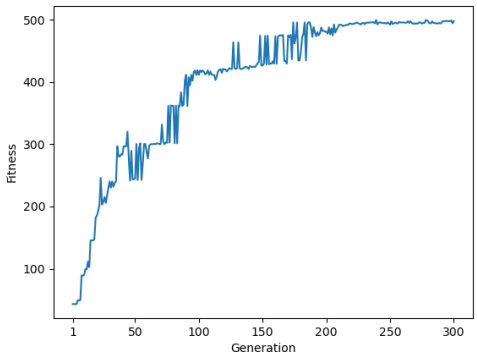

Figure 37. Fitness value of 2D model across generations.

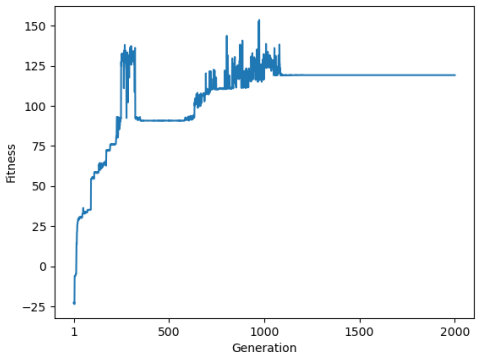

Figure 38. Fitness value of 3D model across generations.

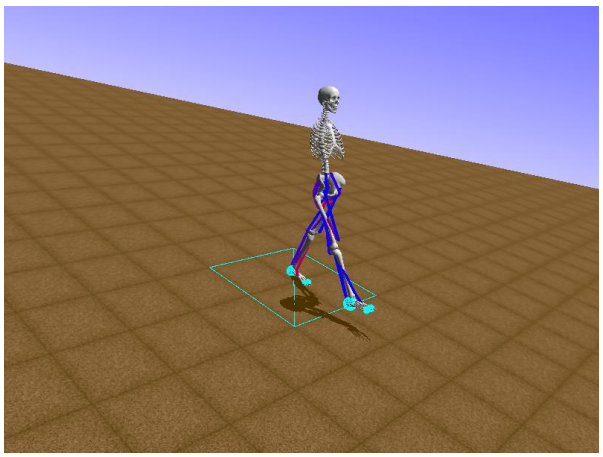

Figure 39. Screenshot of the visualized environment during the 1st step of the 2D model.

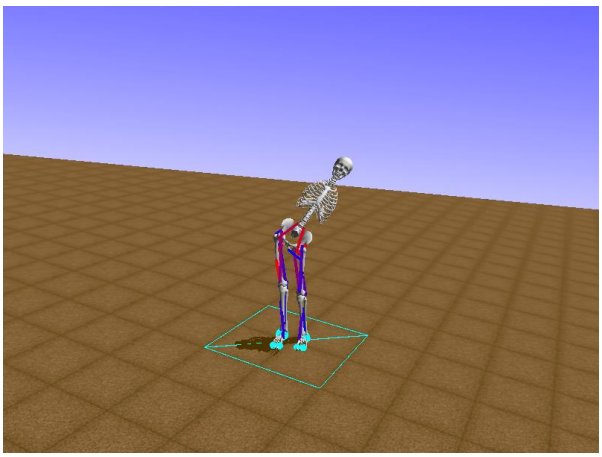

Figure 40. Screenshot of the visualized environment during the 1st step of the 3D model.

The source code of the genetic algorithm can be found in *ga.py.*

4.4.1. Crossover

Crossover is implemented according to 3. Design. Two different walking strategies participating in crossover may have different periods, but the resulting walking strategy should have a single period. For that purpose, when the crossover occurs, I define a new period as a random choice from periods of two strategies, and then convert all the muscles of the resulting strategy to this period.

Crossover should be a meaningful operation that can potentially bring something new to the result. For example, if I take one muscle from one walking strategy and another muscle from another, there is a chance that the resulting walking strategy will be better than the initial two. But if I take some parts of some muscle from one strategy and somehow connect them with some parts of some muscle of another walking strategy, it is not obvious if such crossover can bring in something useful. Therefore, I cross strategies by combining their muscles as indivisible entities but not crossing muscles internals. The crossover of muscle internals requires more profound research to prove its usability.

The source code for crossover can be found in *walking_strategy.walking_strategy.WalkingStrategy.crossover.*

4.4.2. Mutation

Mutation is one of the most sophisticated parts of implementation as it uses a lot of math and should provide consistency of mutation of different parts of the genotype.

Amount of mutation is regulated by dynamic mutation rates changing during execution of the genetic algorithm.

Period, type, sampling interval, and precisions are mutated by choosing from lists of available values. The period should be the same for all the muscles within a single walking strategy, sampling interval and precision should conform to **Equation (8)**.

$$precision \leq \frac{period}{sampling\_interval} \qquad (8)$$

Direct components are mutated by adding a number from the Gaussian distribution with a center of *0* and a standard deviation of *0.1* and clipped to range [0,1] after mutation.

Fourier components are complex numbers but instead of mutating real and imaginary parts just by Gaussian distribution, I mutate their amplitude and phase. Such an approach creates a simple latent space for mutations and improves the learning rate, see **Figure 41** for comparison.

Amplitude is mutated by calculating a unit vector of a complex number, multiplying it to a mutation factor, and transforming it back to a complex number. The mutation factor is a random number from a normal distribution with a center of 0 and a standard deviation of $\frac{period}{sampling\_interval}$ and clipped to range $[-0.5 \cdot \frac{period}{sampling\_interval}, 0.5 \cdot \frac{period}{sampling\_interval}]$ after mutation.

Phase is mutated by multiplying a complex number by $e^{i \cdot mutation\_factor}$, where *mutation_factor* is a random number from a normal distribution with a center of *0* and a standard deviation of $\frac{\pi}{2}$ and clipped to range $[-\frac{\pi}{2}, \frac{\pi}{2}]$ after mutation.

For activation values to be in the range [0, 1] after the mutation of Fourier components, I perform IFFT after each mutation of amplitude and calculate the normalization coefficient for specific mutated amplitude. Though such an approach does not provide strict inclusion to the range, the outliers usually exceed the range by not more than 30%, which is enough to achieve results.

Figure 41. Fitness value across generations for mutation with and without latent space.

Figure 42 shows examples of muscle activation before and after mutations.

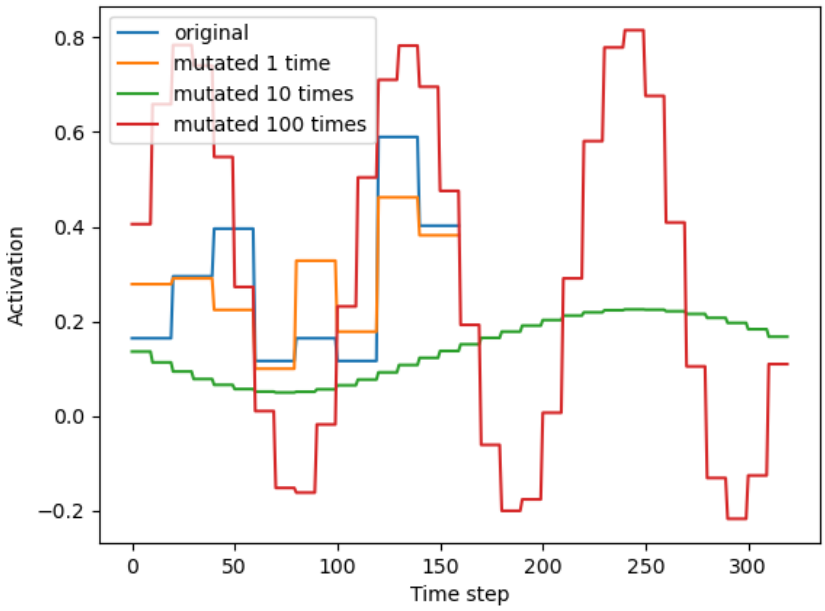

Figure 42. Example of how activation of the same muscle changes after mutations.

The source code for mutation can be found in *walking_strategy.walking_strategy.WalkingStrategy.mutate* and *walking_strategy.muscle.Muscle.*

4.4.2. Elitism

During crossover and mutation, the best-performing instances of walking strategies can be mutated into something worse and the progress will be lost. To avoid such an effect, I preserve a part of the population with the best fitness values for the next generation. [1] calls it “a survival-ratio“ and preserves 20% of the population. I compared different percentages of elites and also chose to preserve 20% as it balances absolute values of learning rate and its stability. See the comparison in **Figure 43**.

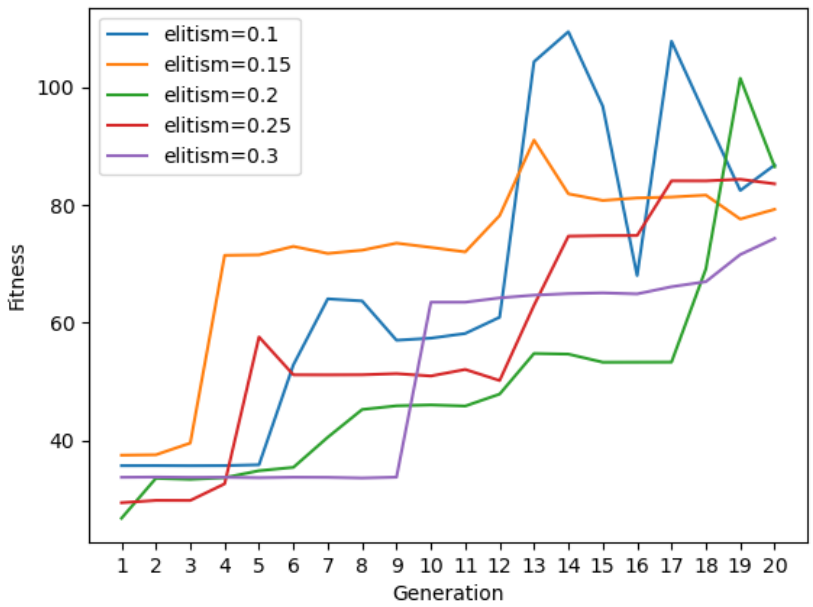

Figure 43. Examples of dependency of fitness on generation for different ratios of elites.

The source code for elitism can be found in *walking_strategy.population_evaluator.PopulationEvaluator.breed_new_population.*

4.5. Simulation environment

The project uses the latest version of OpenSim [5] at the time of implementation (*4.5.1*) and the models described in 3. Design. The model format is also updated to the latest version using OpenSim UI.

To speed up the simulation and evaluation of the population, I use Python multiprocessing and run the simulation environment in 30 parallel processes. This number is chosen because it is close to the CPU parallelization limit described in 3. Design and allows for reducing the execution time to 1 minute, as expected in evaluation criteria.

Integrator accuracy is an OpenSim configuration option that allows for managing the amount of calculational error during simulation. I set it to *0.001* as it provides a balance between simulation duration and progress of the fitness value and is mentioned in [20] as a practically proven stable value. See **Figures 44, 45** for the comparison of different integrator accuracy.

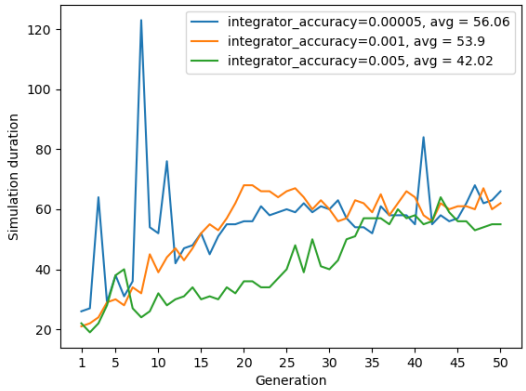

Figure 44. 2D-simulation duration of a single population across generations for different integrator accuracy.

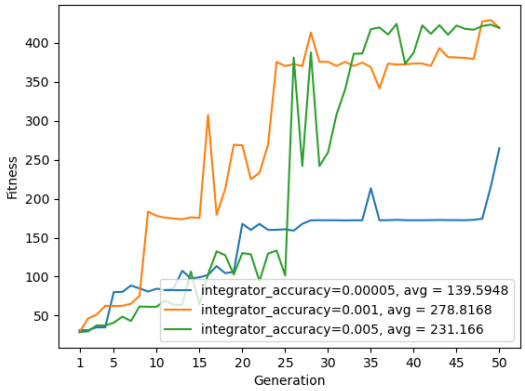

Figure 45. Fitness of 2D-model across generations for different integrator accuracy.

Sometimes, OpenSim freezes for no obvious reason, and the simulation duration increases hundreds of times. To prevent that, I set a duration limit of 20 seconds for a single simulation.

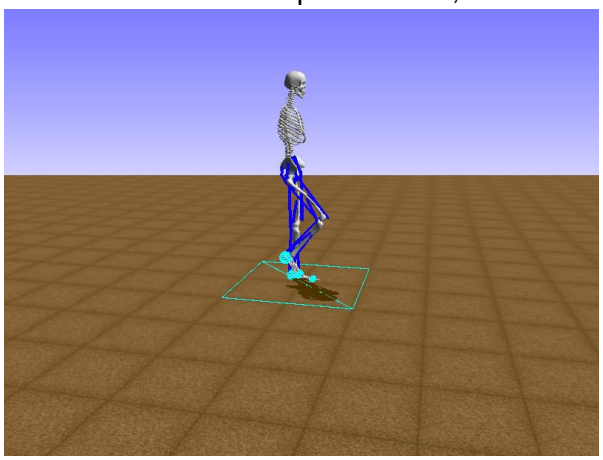

Figure 46. Initial pose in 2D simulation.

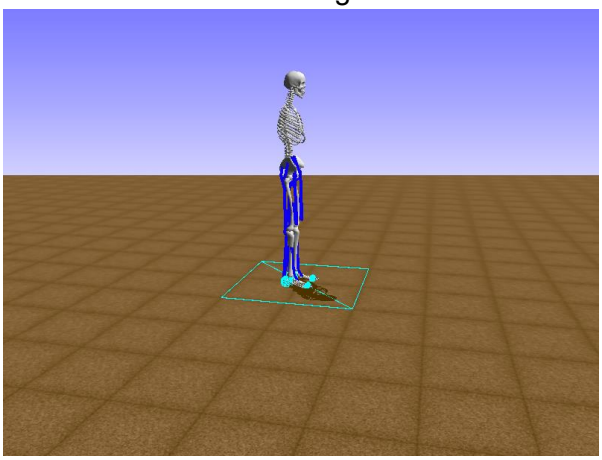

Figure 47. Initial pose in 3D simulation.

Figures 46, 47 show initial model poses for 2D and 3D models. To ease learning, I slightly moved the right leg of the 2D model to a pose more similar to a position between steps. However, that does not work for 3D, as the model fails on the floor faster from this initial pose.

The simulation environment configuration options include 2D or 3D modes and whether or not to visualize the simulation.

The source code for the simulation environment can be found under the *sim* directory.

4.6. Performance optimization

Main logical and technical optimizations include:

1. Perlin noise for initial generation as described in 4.1 Initial population.
2. Multiprocessing for simulation as described in 4.5 Simulation environment.
3. Adjusting integrator accuracy as described in 4.5 Simulation environment.
4. Using numpy [17] for optimized mathematical calculations: *np.fft*, *np.ifft*, *np.random.choice*, and *array* operations.
5. Implementing latent space for mutation as described in 4.4.2 Mutation.
6. Simulation time limit as described in 4.5 Simulation environment.
7. Adjusted initial pose as described in 4.5 Simulation environment.

4.7. Testing

I used two methods for testing the implementation:

1. Unit tests to automatically minimize risks of typos, calculation errors, and logical bugs.
2. Manual visual tests using plots and environment visualization. I built and verified plots of muscle activation for the encoding scheme, mutations, and evolved muscles to see what was happening and modify algorithms if needed. Periodical visual checks during learning allowed for timely stopping of the learning, implementing fixes, and restarting the process if something went critically wrong, for example, with the fitness function.

The source code for both methods can be found under the *tests* directory. The unit tests coverage report is available in *htmlcov/index.html.*

5. Evaluation

The 1st objective is successfully achieved: the model walks in 2D simulation throughout the entire allotted time imitating a real human walking. **Figures 32, 39** and the video show the achieved gait. The 2nd object is partially achieved: though the 3D model can walk, it makes only 4 steps and falls afterward. **Figure 40** and the video show the achieved gait.

Among the successful moments, I can highlight the implementation of a genetic algorithm that trains a 2D model to walk just for 225 generations and about 4 hours. Perlin noise for initial generation and the latent space for mutation were the key ingredients that dramatically improved the learning performance (see **Figures 27, 41**). However, the hypothesis that working 2D walking strategy can be reused as an initial generation for training 3D walking was wrong: 2D walking strategy in 3D simulation is trained slower and falls faster than 3D walking strategy from scratch after 2-3 generations. It seems that 2D and 3D walking have principal differences in muscle activations regarding stability.

Evaluation Criteria			2D	3D
Unit test coverage, *%*			79	
Duration of simulation of a single population, *seconds*	Avg		54.47	30.97
	Min		24.0	10.0
	Max		69.0	42.0
	Mdn		56.0	36.0
Duration of evaluation and generation of a new population, *seconds*	Avg		1.95	1.83
	Min		1.0	1.0
	Max		3.0	3.0
	Mdn		2.0	2.0
Walking stability, *simulation steps*			1000	395
Distance, *meters*			11.15	1.9
Energy consumed per 1 meter of the distance traveled, *conventional units*			2.62	7.94
Subjective visual comparison of gait with top solutions from NIPS "Learn to Move" challenges			Achieved 2D walking is similar to natural human walking with bending knees and noticeable step sizes, while the 3D model does not bend the knees much and makes small steps. In [12, 13, 14], a model walks without periodically tilting the torso for 2D and 3D cases. However, 2D in my project noticeably tilts the torso forward and backward periodically, and the 3D model tilts it to the left and right. Unlike the models from my project, all the models in [12, 13, 14] walk by slightly tilting their torsos to the front, as if falling, and putting their legs up so as not to fall entirely.	
Learning rate	Increase in fitness per generation	Avg	2.2	0.11
		Min	-61.2	-43.58
		Max	60.96	37.78

		Mdn	0.42	0.01
	Last achieved fitness		497.84	119.14

Table 3. Evaluation results.

Table 3 contains evaluated criteria from 3.7. Testing and evaluation plan. Learning rate is measured until the generation when the fitness value becomes steady: *225* for 2D and *1100* for 3D. **Figures 48-57** show how evaluation values changed during training. Achieved unit tests coverage of 79% satisfies the evaluation criteria which allows for decreasing the amount of bugs. The 2D model performs as expected, while 3D shows worse evaluation metrics. One main reason is that the 2D model has *9* degrees of freedom, while 3D has *14*. That difference introduces the whole new task of balancing: while 2D can only fall forward or backward, 3D has an infinite number of falling directions, which inflates the exploration space dramatically. For the same reason the 3D model spends more energy than 2D. Simulation duration increases with generation number as a model improves and lives longer. 2D has a longer simulation duration as it has higher walking stability and lives more simulation steps. Both simulation and evaluation durations achieved satisfy to initial evaluation criteria.

To check that the results achieved were not lucky coincidences, I reproduced the learning process from scratch three times for both 2D and 3D models and obtained similar results.

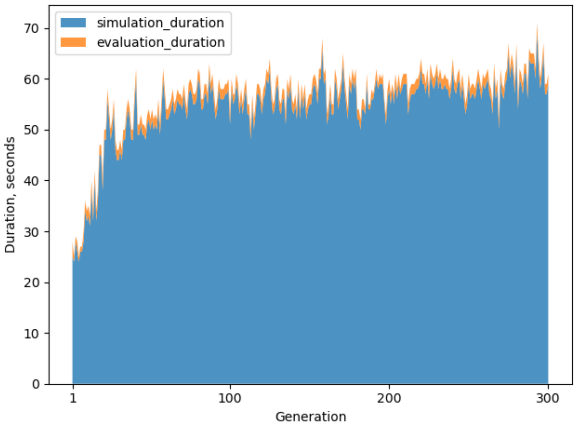

Figure 48. Duration of simulation and evaluation of a single population across generations for 2D.

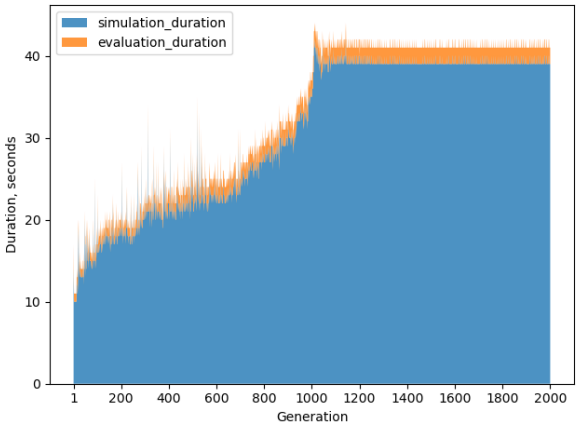

Figure 49. Duration of simulation and evaluation of a single population across generations for

Monitoring the learning rate is crucial for success. I found heuristically that if the fitness value does not improve or change for even tens and all the more so hundreds of generations, something is probably wrong in the implementation.

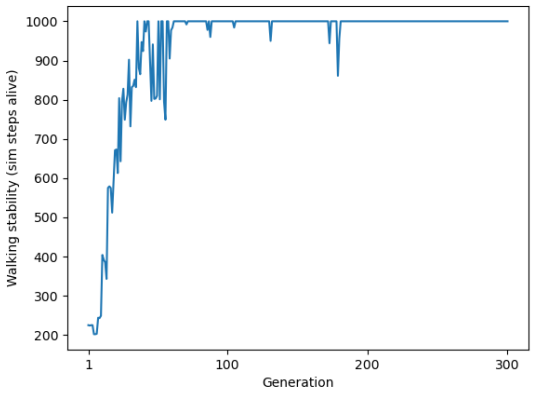

Figure 50. Walking stability of a walking strategy with the best fitness in the population across generations for 2D.

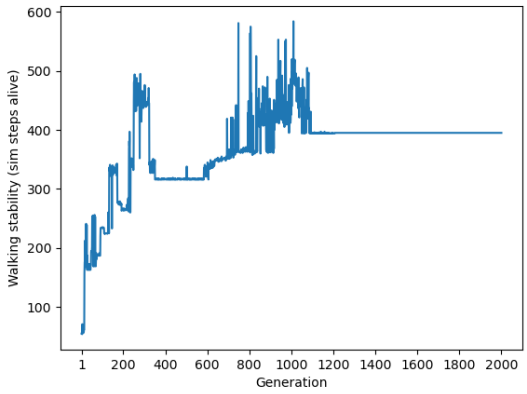

Figure 51. Walking stability of a walking strategy with the best fitness in the population across generations for 3D.

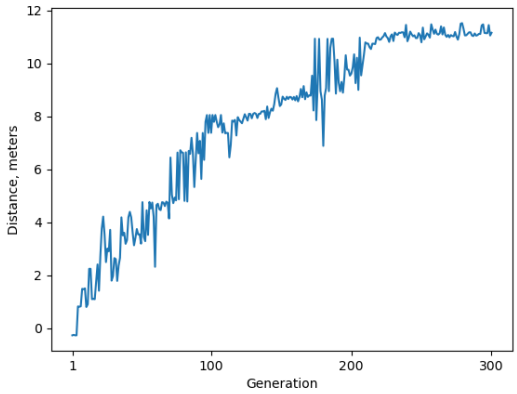

Figure 52. Distance passed by a walking strategy with the best fitness in the population across generations for 2D.

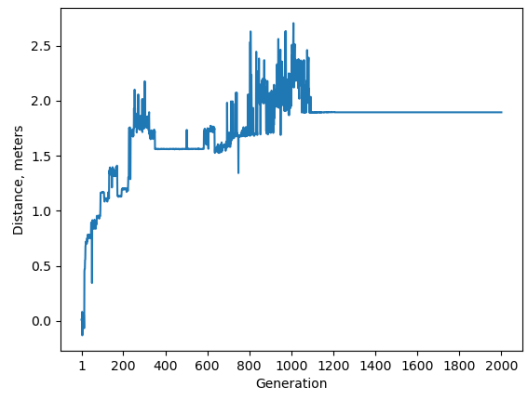

Figure 53. Distance passed by a walking strategy with the best fitness in the population across generations for 3D

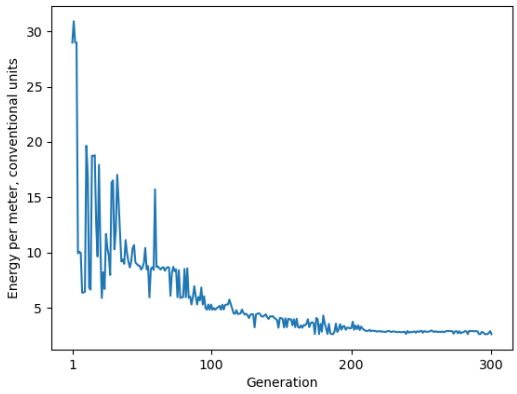

Figure 54. Energy per a meter spent by a walking strategy with the best fitness in the population across generations for 2D.

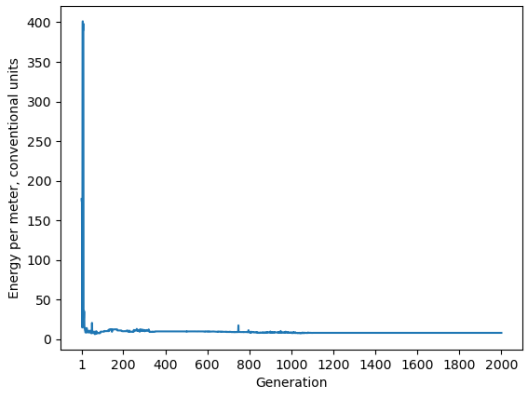

Figure 55. Energy per a meter spent by a walking strategy with the best fitness in the population across generations for 3D.

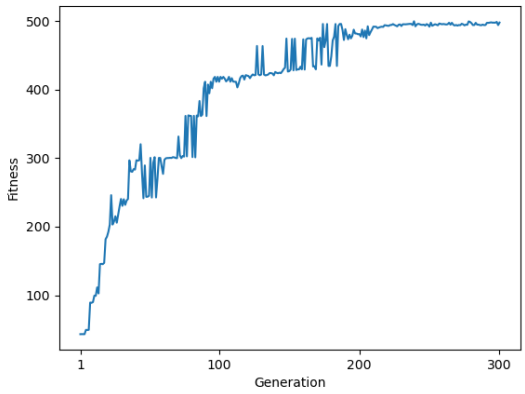

Figure 56. Best fitness value in the population across generations for 2D.

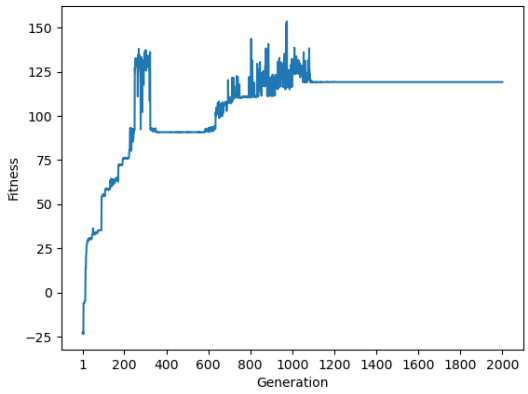

Figure 57. Best fitness value in the population across generations for 3D.

Initial project objectives included experimenting with sensors and disabled muscles, but during the prototype and development phases, I found that these aims may be too ambitious and require at least twice the volume of work and text to describe. So, I excluded them from the scope of this project. Otherwise, the designed plan corresponds to the actual progress of the project.

There were two main limitations:

1. CPU resources. 4.6. Performance optimization describes how I addressed them.
2. The OpensSim library has detailed documentation for C++ but not for Python. At the same time, the C++ and Python versions have many differences. I used examples from [6] and the library source code from [21, 22] to experiment with the library and solve emerging tasks.

Novelty aspects include:

1. Implementing a latent space for mutation of Fourier components. Though such transformations are not new, no research papers mention using such an approach for human walking. This is my idea in the context of this project.
2. Random generation of the initial generation using Perlin noise. Such an approach is also not mentioned anywhere and is my idea.
3. There is no public research about applying genetic algorithms to human walking using muscles. Existing papers with genetic algorithms use 6-8 joints instead of 22 muscles, and the papers that work with muscles use reinforcement learning. So, my project demonstrates the effective applicability of genetic algorithms to muscles, which is also closer to real-world examples.
4. Many papers dedicated to training bipedal locomotion operate in 2D space, especially those with genetic algorithms. My project also extensively experiments with 3D space and achieves meaningful results.

Possible extensions may be:

1. Continue experimenting with the 3D environment. It looks like balance and stability become highly important in 3D space, and probably, simple periodic walking does not work already. It should be modulated by some signals from balance sensors. Also, more advanced latent space is probably required for 3D to improve learning performance.
2. Disable muscles or add prosthetic legs to experiment with how the gait changes and evolves. This aspect has more practical application.
3. Research option to add random seed to stochastic parts of the algorithm for the sake of deterministic reproducibility. I currently did not implement the random seed as the simulation

environment has lowered precision, and even the same random seed may provide different results and not work as expected.

As an element of self-evaluation, I would mention that initially, the project seemed more trivial. Still, many complexities arose during the work: I spent much time tuning the genetic algorithm and optimizing the learning process. As a result, the original scope was changed and simplified. Now, I think that when starting a project, it is always a good idea to estimate risks and prepare one or more fallback plans explicitly. It is good that I started working on the project on time, according to the plan, instead of doing everything at the last moment. Otherwise, I would fail the project as making a model to walk in 3D space turned out to be much more complicated than 2D. Another tricky part of the project is length limits, as I have a lot of significant details to describe and a lot of analytics to present. While working on the project, I acquired more practical experience with genetic algorithms, learned about OpenSim and how to work with it, enhanced my understanding of Fourier transform, and realized the importance of latent space for mutations. The areas for improvement may be deeper research of technical optimizations in OpenSim, collecting and using analytics from the start of the project instead of its middle or final parts, and more profound preliminary research of genetic algorithms to avoid known pitfalls.

6. Conclusion

During the project, I experimented with different configurations and implementations of the encoding scheme, the genetic algorithm, mutability, and crossing approaches to discover muscle contraction curves optimal for human gait. Achieving the desired walking in 2D was much easier than in 3D. It looks like any mutability approach that covers all the possible combinations can lead to the solution, but it is mostly a question of time. It is important to have at least a basic understanding of the domain to discover and apply simplifications, optimizations, and so on to reduce learning time and increase overall performance.

Some possible extensions are mentioned in 5. Evaluation. Broader further work may also include experiments with the more advanced “brain” part of the model with mapping and transformations between sensor inputs and muscle outputs. However such work requires the manual design of the “brain”, while Deep Reinforcement Learning provides similar capabilities with fewer requirements for the manual design.

Genetic Algorithms are good for searching when we do not know exactly how the final result looks or when there are many suitable results. But they are not very optimal when we expect a single correct solution, as it is hard to find it in an almost infinite space of combinations. For tasks when we know how the final result should look and there are more strict reward criteria, Deep Reinforcement Learning is expected to show better results.

Nevertheless, I showed in this project that Genetic Algorithms are applicable and have many perspectives in the context of human walking. Many research papers also use Genetic Algorithms as a good option for finding an optimal architecture and hyperparameters in neural networks. And they can be used for faster search of a good initial model/simulation configuration with decent reward in Reinforcement Learning instead of total random.

7. References

[1] Sims, Karl. *Evolving virtual creatures*. Proceedings of the 21st annual conference on Computer graphics and interactive techniques, Orlando, Florida, United States of America, 1994, Association for Computing Machinery, New York, NY, United States, Pages 15–22.

[2] Song, S., Kidziński, Ł., Peng, X.B. et al. Aug 16, 2021. Deep reinforcement learning for modeling human locomotion control in neuromechanical simulation. Journal of NeuroEngineering and Rehabilitation, 18, 126 (2021).

[3] Kidziński, Ł. et al. Apr 2, 2018. *Learning to Run challenge solutions: Adapting reinforcement learning methods for neuromusculoskeletal environments*. ArXiv, 1804.00361.

[4] Song, S., Kidziński, Ł., et al. Feb 2, 2020. Reinforcement learning with musculoskeletal models in OpenSim. Reinforcement learning with musculoskeletal models. https://osim-rl.kidzinski.com/.

[5] National Institutes of Health (NIH). Jun 5, 2024. OpenSim. SimTK: OpenSim: Project Home. https://simtk.org/projects/opensim/.

[6] Song, S., Kidziński, Ł., et al. Feb 2, 2020. stanfordnmbl/osim-rl. GitHub - stanfordnmbl/osim-rl: Reinforcement learning environments with musculoskeletal models. https://github.com/stanfordnmbl/osim-rl/tree/master

[7] Norman Di Palo. Oct 11, 2017. Learning to walk with evolutionary algorithms applied to a bio-mechanical model. Learning to walk with evolutionary algorithms applied to a bio-mechanical model | by Norman Di Palo | Towards Data Science. https://towardsdatascience.com/learning-to-walk-with-evolutionary-algorithms-applied-toa-bio-mechanical-model-1ccc094537ce.

[8] Norman Di Palo. Oct 14, 2017. normandipalo/learn-to-walk-with-genetic-algs. GitHub - normandipalo/learn-to-walk-with-genetic-algs: Learning to walk with genetic algorithms by optimizing a truncated Fourier series, applied to a realistic bio-mechanical model of the human body. https://github.com/normandipalo/learn-to-walk-with-genetic-algs.

[9] Sylvester AD, Lautzenheiser SG, Kramer PA. Jul 15, 2021. Muscle forces and the demands of human walking. Biology Open, 10(7):bio058595.

[10] Amjad Yousef Majid, Serge Saaybi, Tomas van Rietbergen, Vincent Francois-Lavet, R Venkatesha Prasad, Chris Verhoeven. Sep 28, 2021. Deep Reinforcement Learning Versus Evolution Strategies: A Comparative Survey. ArXiv, 2110.01411.

[11] Qin Yongliang. Oct 7, 2018. ctmakro/stanford-osrl#The simulation is too slow. GitHub - ctmakro/stanford-osrl: NIPS2017 challenge. https://github.com/ctmakro/stanford-osrl#the-simulation-is-too-slow.

[12] Shu Ha. Nov 4, 3019. performance of 1st place in NeurIPS 2019: Learn to Move challenge. performance of 1st place in NeurIPS 2019: Learn to Move challenge - YouTube. https://www.youtube.com/watch?v=FD2lGv-4BLE.

[13] NNAISENSE. Mar 4, 2018. NIPS 2017 Learning to Run competition | NNAISENSE winning runner. NIPS 2017 Learning to Run competition | NNAISENSE winning runner - YouTube. https://www.youtube.com/watch?v=13uIIU8ACRE.

[14] Łukasz Kidziński. Oct 7, 2017. Learning to run NIPS challenge. Learning to run NIPS challenge - YouTube. https://www.youtube.com/watch?v=mbjuaRg__DI.

[15] Jennifer Hicks. Apr 5, 2024. Scripting in Python. Scripting in Python - OpenSim Documentation - OpenSim. https://opensimconfluence.atlassian.net/wiki/spaces/OpenSim/pages/53085346/Scripting+in+Python.

[16] Python Software Foundation. 2024. Python. Welcome to Python.org. https://www.python.org/.

[17] NumPy team. 2024. NumPy . NumPy -. https://numpy.org/.

[18] Ildar SALAKHIEV. Jan 29, 2022. salaxieb/perlin_noise. GitHub - salaxieb/perlin_noise: Smooth random noise generator. https://github.com/salaxieb/perlin_noise.
[19] The SciPy community. 2024. SciPy API → Interpolation (scipy.interpolate) → interp1d. interp1d — SciPy v1.14.0 Manual. https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.interp1d.html.
[20] Jan Ebert. Jul 11, 2018. Too low integrator accuracy leads to exception. Too low integrator accuracy leads to exception · Issue #138 · stanfordnmbl/osim-rl · GitHub. https://github.com/stanfordnmbl/osim-rl/issues/138.
[21] National Institutes of Health (NIH). Aug 17, 2024. opensim-org/opensim-core. GitHub - opensim-org/opensim-core: SimTK OpenSim C++ libraries and command-line applications, and Java/Python wrapping. https://github.com/opensim-org/opensim-core.
[22] National Institutes of Health (NIH). Jul 23, 2024. simbody/simbody. GitHub - simbody/simbody: High-performance C++ multibody dynamics/physics library for simulating articulated biomechanical and mechanical systems like vehicles, robots, and the human skeleton. https://github.com/simbody/simbody.